



NAME: Chandler Matere Simiyu

ADMISSION NUMBER: BSCCS/2024/58571

COURSE: Bachelor of Science in Computer Science

UNIT CODE: BIT4138

UNIT NAME: Advanced Cryptography

LECTURER: Mr. Nyoro Michael

Year III, Semester II

PROJECT TITLE: Cryptanalysis & Security Testing Toolkit

GITHUB LINK: <https://github.com/Simiyuu/cryptanalysis-toolkit>

WEEK 1: Cryptography Environment Setup.....	5
Installation and Configuration of Cryptography Development Environment.....	5
Fig 1: Python 3.14.3 and VS Code confirmed installed and configured as the primary development environment for the Cryptanalysis and Security Testing Toolkit.....	5
Fig 2: PyCryptodome and Matplotlib libraries successfully installed via pip, enabling cryptographic implementations.....	6
Fig 3: OpenSSL version confirmed, validating successful installation for generating secure cryptographic keys.....	6
Fig 4: Caesar cipher script executed successfully in VS Code terminal, demonstrating the first working encryption module of the toolkit.....	7
Fig 5: Initial commit pushed to GitHub repository, establishing version control for the Cryptanalysis and Security Testing Toolkit project.....	8
Student Reflection.....	8
Week 1 Summary.....	8
Week 2: Classical Cryptography and Cipher Design.....	9
Implementation of Classical Encryption Algorithms.....	9
Fig 1: Caesar cipher implementation displayed in VS Code, showing the shift-based substitution encryption logic forming the first module of the toolkit.....	9
Fig 2: Vigenère cipher encryption process executed successfully, demonstrating polyalphabetic substitution using a keyword to improve security over Caesar cipher.....	10
Fig 3: Full encryption and decryption cycle confirmed, showing the original message, encrypted ciphertext, and successfully recovered plaintext in one output.....	11
Fig 4: User input validation interface executed, demonstrating menu-driven cipher selection with input validation ensuring correct shift values and message formatting.....	12
Fig 5: Cipher testing results confirmed, showing Caesar cipher successfully encrypting and decrypting multiple test cases with consistent and accurate outputs.....	13
Student Reflection.....	13
Week 2 Summary.....	13
Week 3: Stream Ciphers and Randomness Testing.....	14
Implementation of Stream Ciphers Systems and Random Sequence Analysis.....	14
Fig 1: LFSR generator implementation displayed in VS Code, showing the feedback logic and tap configuration used to generate pseudorandom binary sequences.....	14
Fig 2: Pseudorandom binary sequence successfully generated using the LFSR, displaying seed value, tap positions, and the resulting 20-bit output sequence.....	15
Fig 3: Statistical randomness tests executed, with Frequency, Runs and Mean tests all returning PASS results confirming strong randomness in the generated sequence.....	16
Fig 4: RC4 stream cipher simulation executed successfully, displaying the keystream generation, byte-level ciphertext output and confirmed decryption of the original message.....	17
Fig 5: Encryption performance test results showing Caesar cipher and RC4 execution times, confirming RC4 is significantly faster for encrypting larger data sets.....	18
Student Reflection.....	18
Week 3 Summary.....	18
Week 4: Block Cipher Design and AES.....	19

Advanced Encryption Standard (AES) and Block Cipher Operations.....	19
Fig 1: AES encryption script displayed in VS Code, showing CBC mode implementation with key configuration and padding logic for secure block encryption.....	19
Fig 2: AES key generation process executed, displaying randomly generated 128-bit, 192-bit and 256-bit keys confirming secure key creation using PyCryptodome.....	20
Fig 3: File encryption demonstrated successfully, showing plaintext read from secret.txt, encrypted using AES CBC mode and saved as an encrypted binary file.....	21
Fig 4: AES decryption results confirmed, showing the encrypted binary file successfully decrypted using the correct key and original plaintext message fully recovered.....	22
Fig 5: AES performance test results displayed, showing encryption times across four message sizes confirming AES maintains strong and consistent performance at scale.	23
Student Reflection.....	23
Week 4 Summary.....	24
Week 5: Public Key Cryptography (RSA).....	24
Implementation of RSA Encryption and Key Management.....	24
Fig 1: RSA key pair generation script displayed in VS Code, showing 2048-bit key generation logic with public and private keys saved as PEM files.....	24
Fig 2: RSA public key encryption process executed, showing the original message encrypted using the public key and outputting a secure hexadecimal ciphertext.....	25
Fig 3: RSA private key decryption confirmed successful, showing the encrypted ciphertext decrypted using the private key and original message fully recovered.....	26
Fig 4: Secure message transmission simulated, demonstrating sender encrypting with receiver's public key and receiver successfully decrypting using their private key.....	27
Fig 5: RSA testing and validation suite executed, with key generation, encryption validation and performance tests all returning PASS confirming a fully functional RSA implementation.....	28
Student Reflection.....	28
Week 5 Summary.....	29
Creativity Section: Frequency Analysis Visualizer.....	29
Letter Frequency Analysis Tool — Cryptanalysis Extension Module.....	29
Fig 1: Frequency Analysis Script.....	29
Fig 2: Terminal Frequency Output.....	30
Fig 3: Matplotlib Bar Chart Visualization.....	30
Creativity Reflections.....	31

WEEK 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

Fig 1: Python 3.14.3 and VS Code confirmed installed and configured as the primary development environment for the Cryptanalysis and Security Testing Toolkit.

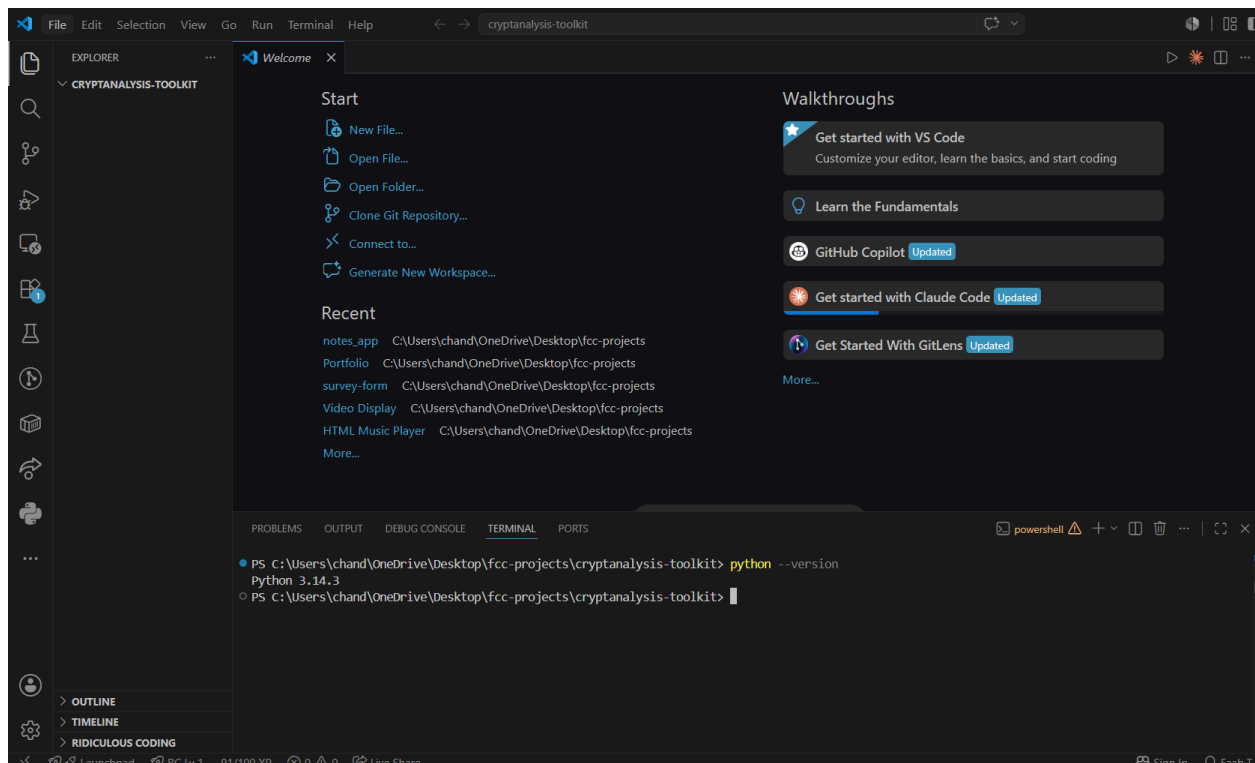


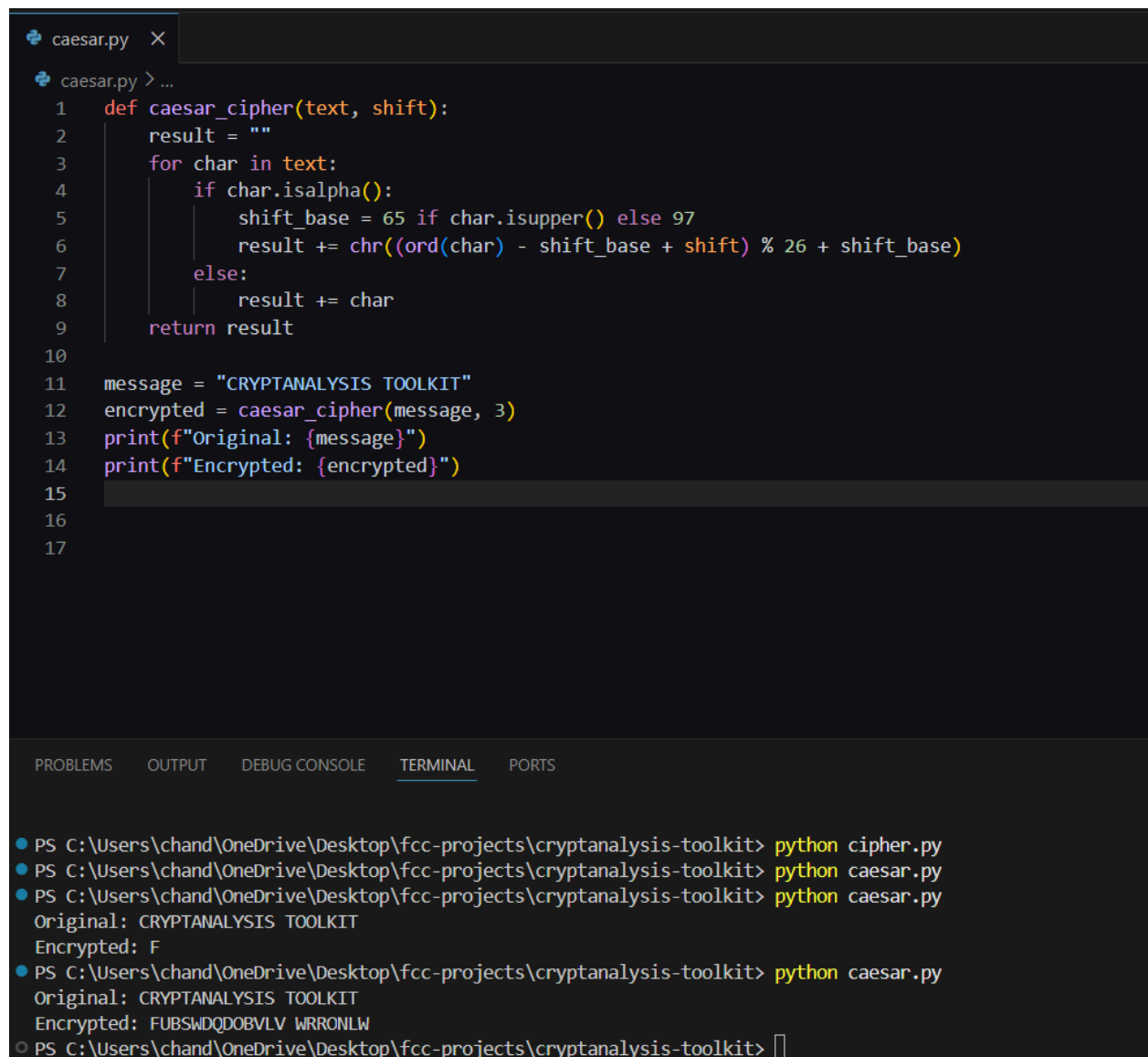
Fig 2: PyCryptodome and Matplotlib libraries successfully installed via pip, enabling cryptographic implementations.

```
C:\Users\chand>pip install pycryptodome matplotlib
Collecting pycryptodome
  Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl.metadata (3.5 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.9-cp314-cp314-win_amd64.whl.metadata (52 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl.metadata (121 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp314-cp314-win_amd64.whl.metadata (5.2 kB)
Collecting numpy>=1.23 (from matplotlib)
  Downloading numpy-2.4.6-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Requirement already satisfied: packaging>=20.0 in .\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in .\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in .\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in .\AppData\Roaming\Python\Python314\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl (1.8 MB)
 1.8/1.8 MB 788.5 kB/s 0:00:04
Downloading matplotlib-3.10.9-cp314-cp314-win_amd64.whl (8.3 MB)
 8.3/8.3 MB 1.7 MB/s 0:00:04
Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl (232 kB)
 232/232 kB 1.7 MB/s 0:00:00
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
 8.3/8.3 kB 1.7 MB/s 0:00:00
Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl (2.3 MB)
 2.3/2.3 MB 1.5 MB/s 0:00:01
Downloading kiwisolver-1.5.0-cp314-cp314-win_amd64.whl (75 kB)
 75/75 kB 1.5 MB/s 0:00:00
Downloading numpy-2.4.6-cp314-cp314-win_amd64.whl (12.5 MB)
12.5/12.5 MB 3.6 MB/s 0:00:03
Installing collected packages: pycryptodome, numpy, kiwisolver, fonttools, cycler, contourpy, matplotlib
 1/7 [numpy] WARNING: The scripts f2py.exe and numpy-config.exe are installed in 'C:\Users\chand\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
 3/7 [fonttools] WARNING: The scripts fonttools.exe, pyftmerge.exe, pyftsubset.exe and ttx.exe are installed in 'C:\Users\chand\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.63.0 kiwisolver-1.5.0 matplotlib-3.10.9 numpy-2.4.6 pycryptodome-3.23.0
```

Fig 3: OpenSSL version confirmed, validating successful installation for generating secure cryptographic keys.

```
MINGW64:/c/Users/chand
chand@Chandler MINGW64 ~
$ openssl version
OpenSSL 3.5.4 30 Sep 2025 (Library: OpenSSL 3.5.4 30 Sep 2025)
chand@Chandler MINGW64 ~
$ |
```

Fig 4: Caesar cipher script executed successfully in VS Code terminal, demonstrating the first working encryption module of the toolkit.



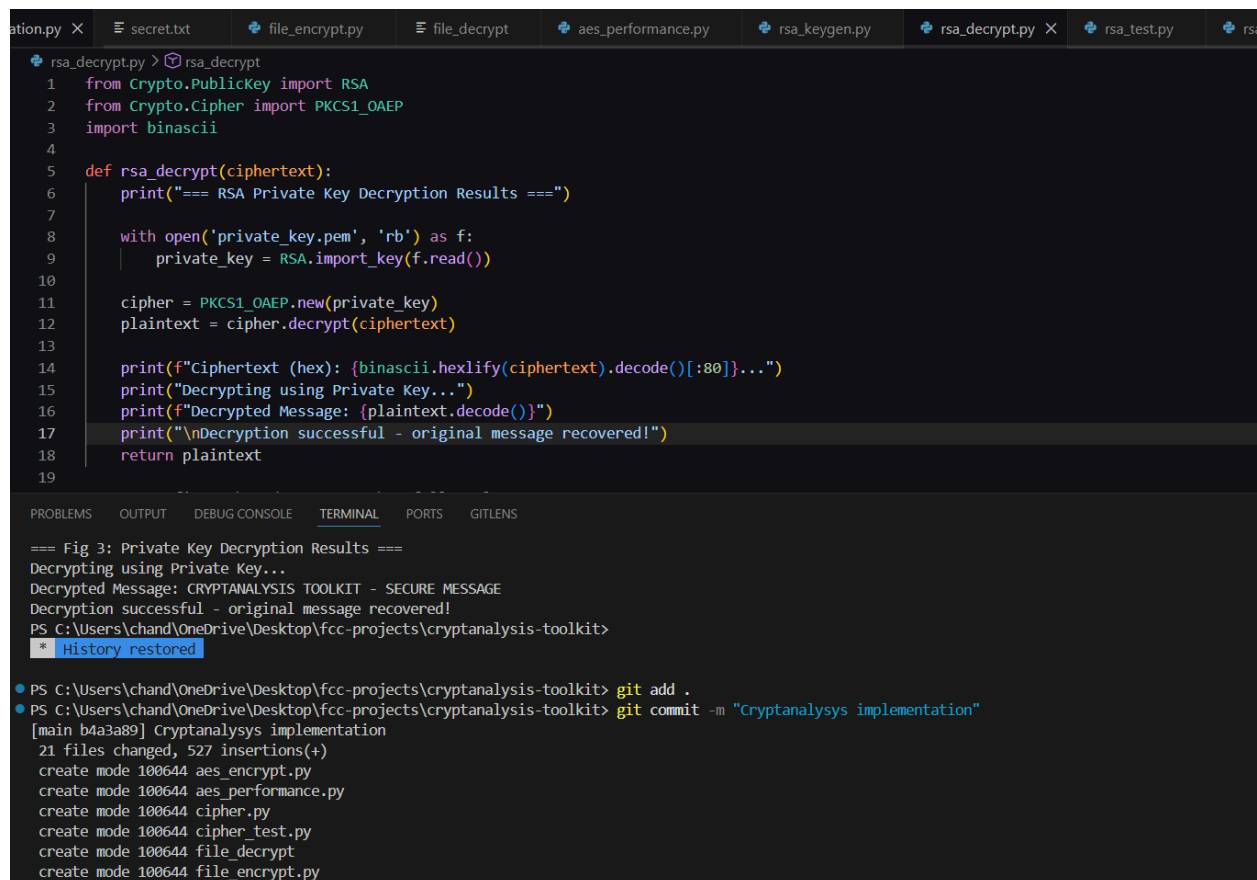
The image shows a Visual Studio Code editor window with a file named `caesar.py`. The script defines a `caesar_cipher` function that takes `text` and `shift` as arguments. It iterates through each character in `text`, checking if it is an alphabet character. If it is, it calculates a new character by shifting its ASCII value by the specified amount, wrapping around the alphabet. If it is not an alphabet character, it leaves the character unchanged. The script then uses this function to encrypt the string "CRYPTANALYSIS TOOLKIT" with a shift of 3, printing both the original and encrypted versions.

```
1 def caesar_cipher(text, shift):
2     result = ""
3     for char in text:
4         if char.isalpha():
5             shift_base = 65 if char.isupper() else 97
6             result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
7         else:
8             result += char
9     return result
10
11 message = "CRYPTANALYSIS TOOLKIT"
12 encrypted = caesar_cipher(message, 3)
13 print(f"Original: {message}")
14 print(f"Encrypted: {encrypted}")
15
16
17
```

The terminal output shows the execution of the script three times, each time using the command `python caesar.py`. The first two executions show the original message "CRYPTANALYSIS TOOLKIT" and the encrypted message "FUBSWDQDOBVLV WRRONLW". The third execution shows the original message "CRYPTANALYSIS TOOLKIT" and the encrypted message "FUBSWDQDOBVLV WRRONLW".

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python cipher.py
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python caesar.py
Original: CRYPTANALYSIS TOOLKIT
Encrypted: F
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python caesar.py
Original: CRYPTANALYSIS TOOLKIT
Encrypted: FUBSWDQDOBVLV WRRONLW
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
```

Fig 5: Initial commit pushed to GitHub repository, establishing version control for the Cryptanalysis and Security Testing Toolkit project.



```
rsa_decrypt.py X secret.txt file_encrypt.py file_decrypt aes_performance.py rsa_keygen.py rsa_decrypt.py X rsa_test.py rs
rsa_decrypt.py > rsa_decrypt
1 from Crypto.PublicKey import RSA
2 from Crypto.Cipher import PKCS1_OAEP
3 import binascii
4
5 def rsa_decrypt(ciphertext):
6     print("=== RSA Private Key Decryption Results ===")
7
8     with open('private_key.pem', 'rb') as f:
9         private_key = RSA.import_key(f.read())
10
11     cipher = PKCS1_OAEP.new(private_key)
12     plaintext = cipher.decrypt(ciphertext)
13
14     print(f"Ciphertext (hex): {binascii.hexlify(ciphertext).decode()[:80]}...")
15     print("Decrypting using Private Key...")
16     print(f"Decrypted Message: {plaintext.decode()}")
17     print("\nDecryption successful - original message recovered!")
18     return plaintext
19
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
=== Fig 3: Private Key Decryption Results ===
Decrypting using Private Key...
Decrypted Message: CRYPTANALYSIS TOOLKIT - SECURE MESSAGE
Decryption successful - original message recovered!
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
* History restored
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> git add .
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> git commit -m "Cryptanalysis implementation"
[main b4a3a89] Cryptanalysis implementation
21 files changed, 527 insertions(+)
create mode 100644 aes_encrypt.py
create mode 100644 aes_performance.py
create mode 100644 cipher.py
create mode 100644 cipher_test.py
create mode 100644 file_decrypt
create mode 100644 file_encrypt.py
```

Student Reflection

This week I configured my cryptography development environment and implemented a Caesar cipher script. Setting up Git and pushing to GitHub proved challenging, particularly resolving the master/main branch conflict. I gained hands-on experience with VS Code, PyCryptodome, OpenSSL, and GitHub, understanding how these tools support secure cryptographic development workflows.

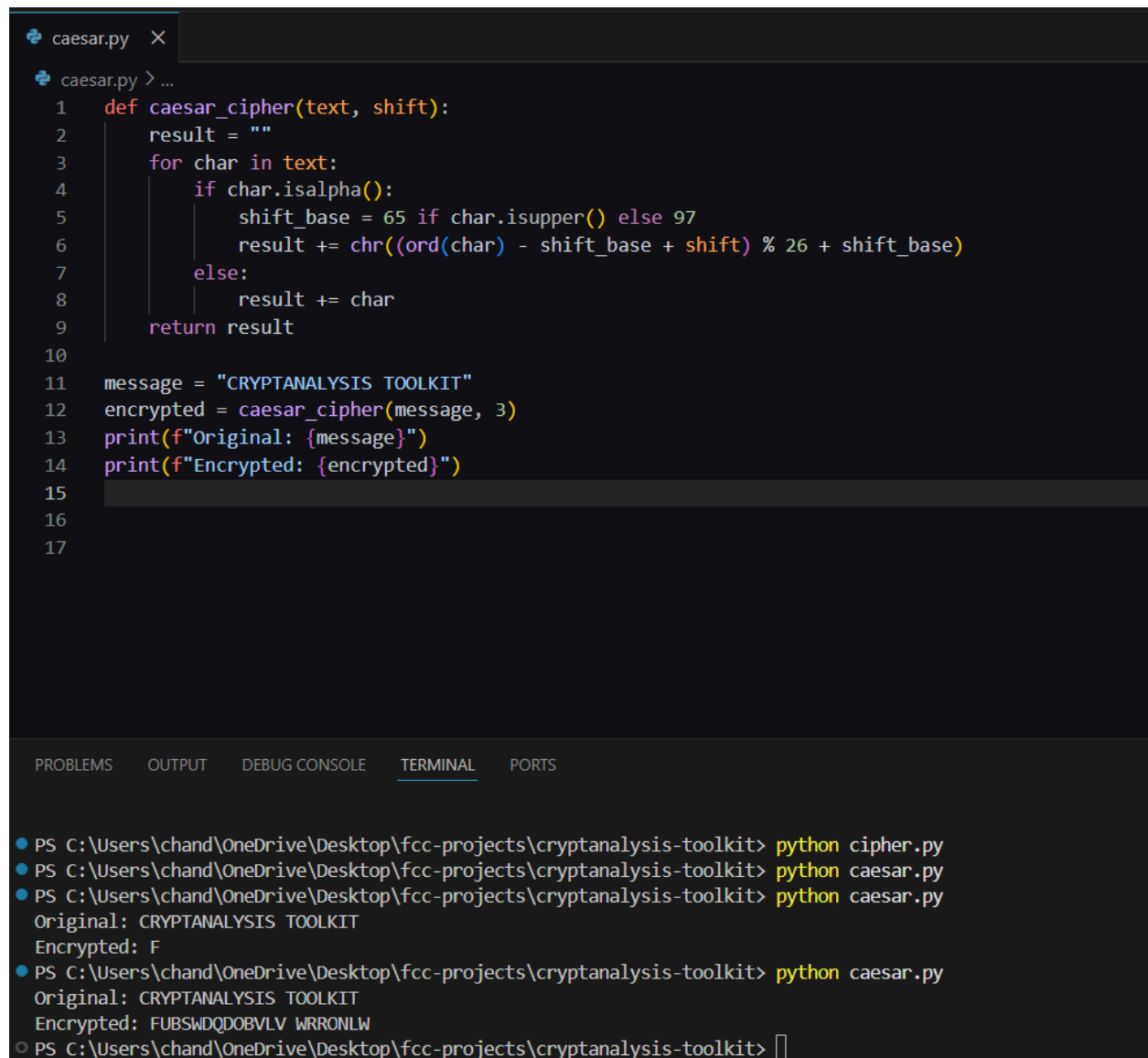
Week 1 Summary

Week 1 involved setting up the cryptography development environment, installing Python, VS Code, OpenSSL and cryptographic libraries. A Caesar cipher script was successfully implemented and pushed to GitHub, establishing the foundation for the Cryptanalysis and Security Testing Toolkit.

Week 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

Fig 1: Caesar cipher implementation displayed in VS Code, showing the shift-based substitution encryption logic forming the first module of the toolkit.

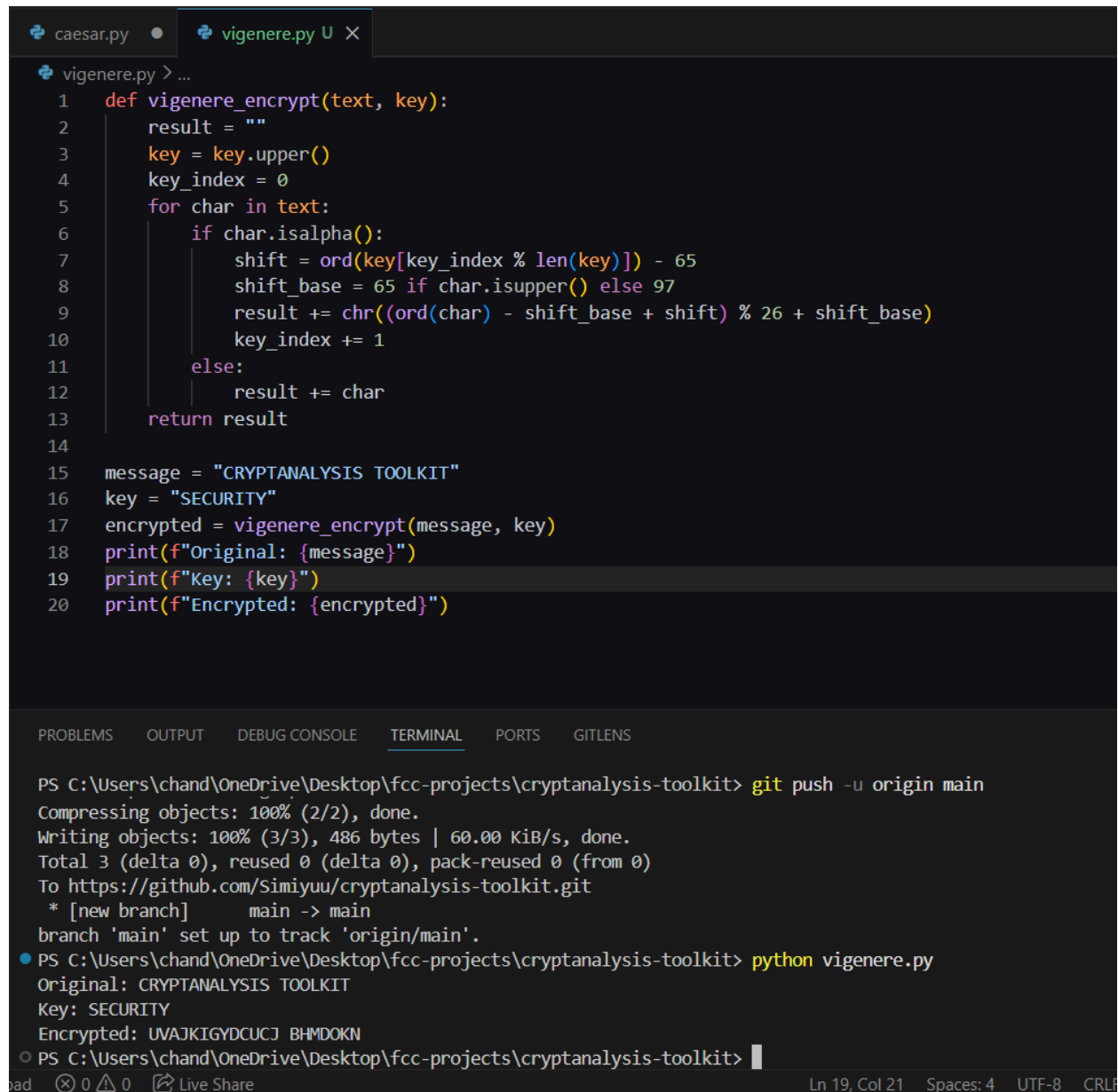


```
caesar.py X
caesar.py > ...
1  def caesar_cipher(text, shift):
2      result = ""
3      for char in text:
4          if char.isalpha():
5              shift_base = 65 if char.isupper() else 97
6              result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
7          else:
8              result += char
9      return result
10
11 message = "CRYPTANALYSIS TOOLKIT"
12 encrypted = caesar_cipher(message, 3)
13 print(f"Original: {message}")
14 print(f"Encrypted: {encrypted}")
15
16
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python cipher.py
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python caesar.py
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python caesar.py
Original: CRYPTANALYSIS TOOLKIT
Encrypted: F
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python caesar.py
Original: CRYPTANALYSIS TOOLKIT
Encrypted: FUBSWDQDOBVLV WRRONLW
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> 
```

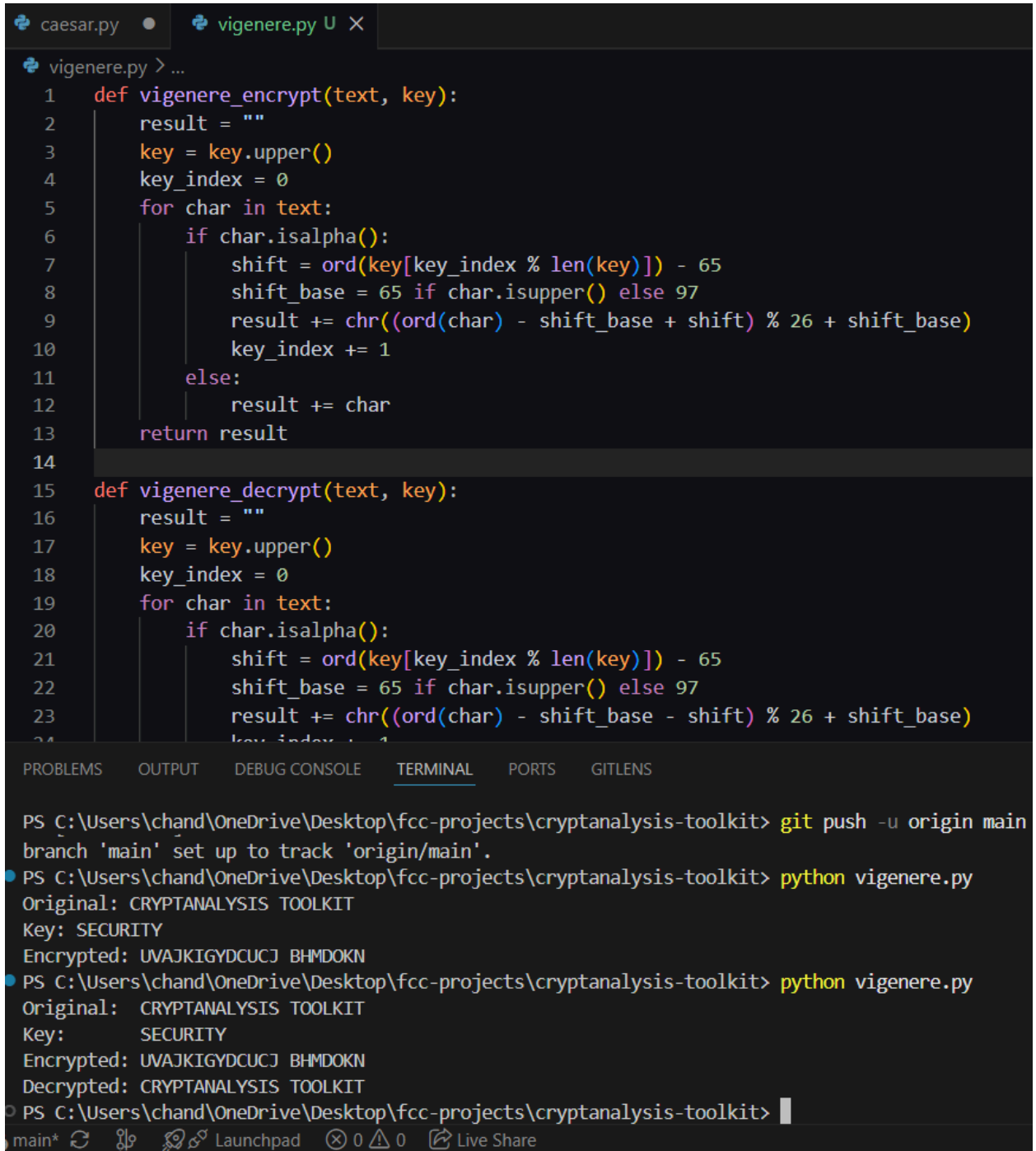
Fig 2: Vigenère cipher encryption process executed successfully, demonstrating polyalphabetic substitution using a keyword to improve security over Caesar cipher.



```
caesar.py • vigenere.py U X
vigenere.py > ...
1 def vigenere_encrypt(text, key):
2     result = ""
3     key = key.upper()
4     key_index = 0
5     for char in text:
6         if char.isalpha():
7             shift = ord(key[key_index % len(key)]) - 65
8             shift_base = 65 if char.isupper() else 97
9             result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
10            key_index += 1
11        else:
12            result += char
13    return result
14
15 message = "CRYPTANALYSIS TOOLKIT"
16 key = "SECURITY"
17 encrypted = vigenere_encrypt(message, key)
18 print(f"Original: {message}")
19 print(f"Key: {key}")
20 print(f"Encrypted: {encrypted}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> git push -u origin main
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 486 bytes | 60.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Simiyuu/cryptanalysis-toolkit.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
• PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python vigenere.py
Original: CRYPTANALYSIS TOOLKIT
Key: SECURITY
Encrypted: UVAJKIGYDCUCJ BHMDOKN
○ PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
bad 0 0 Live Share Ln 19, Col 21 Spaces: 4 UTF-8 CRLF
```

Fig 3: Full encryption and decryption cycle confirmed, showing the original message, encrypted ciphertext, and successfully recovered plaintext in one output.



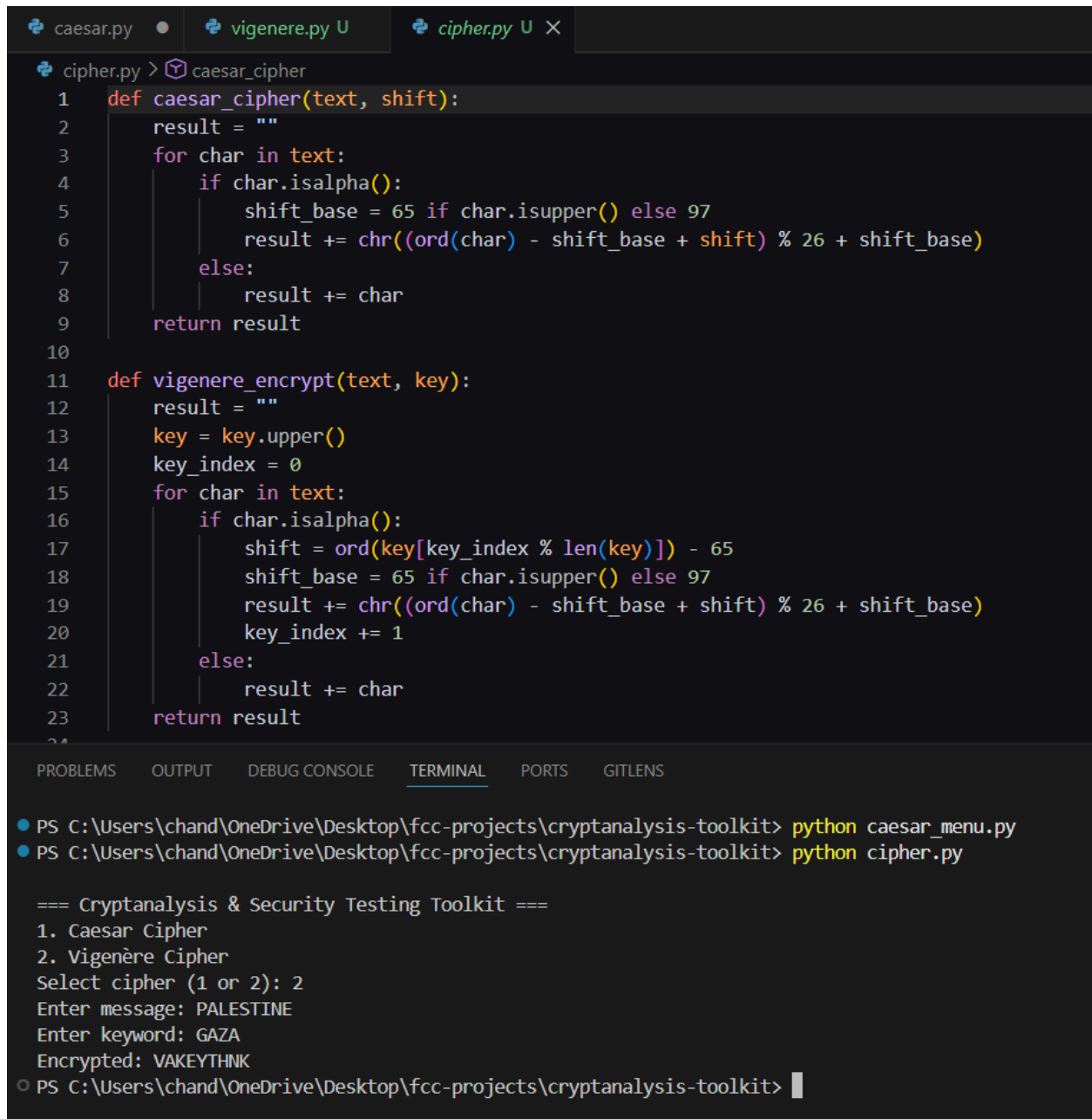
```
caesar.py • vigenere.py U X
vigenere.py > ...
1 def vigenere_encrypt(text, key):
2     result = ""
3     key = key.upper()
4     key_index = 0
5     for char in text:
6         if char.isalpha():
7             shift = ord(key[key_index % len(key)]) - 65
8             shift_base = 65 if char.isupper() else 97
9             result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
10            key_index += 1
11        else:
12            result += char
13    return result
14
15 def vigenere_decrypt(text, key):
16     result = ""
17     key = key.upper()
18     key_index = 0
19     for char in text:
20         if char.isalpha():
21             shift = ord(key[key_index % len(key)]) - 65
22             shift_base = 65 if char.isupper() else 97
23             result += chr((ord(char) - shift_base - shift) % 26 + shift_base)
24            key_index += 1
25        else:
26            result += char
27    return result
28
29 if __name__ == '__main__':
30     original = "CRYPTANALYSIS TOOLKIT"
31     key = "SECURITY"
32     encrypted = vigenere_encrypt(original, key)
33     decrypted = vigenere_decrypt(encrypted, key)
34     print(f"Original: {original}")
35     print(f"Key: {key}")
36     print(f"Encrypted: {encrypted}")
37     print(f"Decrypted: {decrypted}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> git push -u origin main
branch 'main' set up to track 'origin/main'.
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python vigenere.py
Original: CRYPTANALYSIS TOOLKIT
Key: SECURITY
Encrypted: UVAJKIGYDCUCJ BHMDOKN
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python vigenere.py
Original: CRYPTANALYSIS TOOLKIT
Key: SECURITY
Encrypted: UVAJKIGYDCUCJ BHMDOKN
Decrypted: CRYPTANALYSIS TOOLKIT
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
```

main*    Launchpad  0  0  Live Share

Fig 4: User input validation interface executed, demonstrating menu-driven cipher selection with input validation ensuring correct shift values and message formatting.

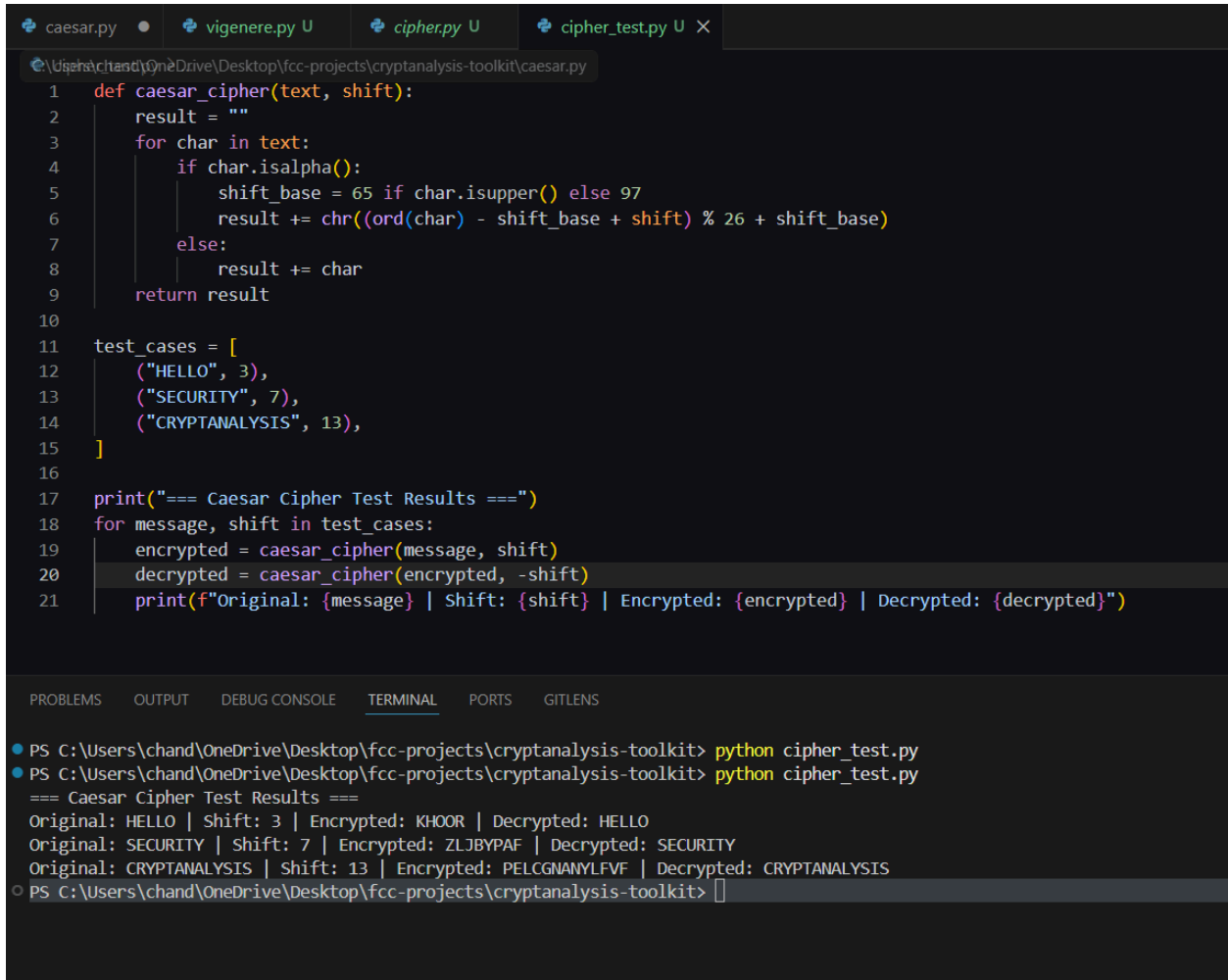


```
caesar.py • vigenere.py U cipher.py U X
cipher.py > caesar_cipher
1 def caesar_cipher(text, shift):
2     result = ""
3     for char in text:
4         if char.isalpha():
5             shift_base = 65 if char.isupper() else 97
6             result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
7         else:
8             result += char
9     return result
10
11 def vigenere_encrypt(text, key):
12     result = ""
13     key = key.upper()
14     key_index = 0
15     for char in text:
16         if char.isalpha():
17             shift = ord(key[key_index % len(key)]) - 65
18             shift_base = 65 if char.isupper() else 97
19             result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
20             key_index += 1
21         else:
22             result += char
23     return result
24

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
● PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python caesar_menu.py
● PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python cipher.py

=== Cryptanalysis & Security Testing Toolkit ===
1. Caesar Cipher
2. Vigenère Cipher
Select cipher (1 or 2): 2
Enter message: PALESTINE
Enter keyword: GAZA
Encrypted: VAKEYTHNK
○ PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> 
```

Fig 5: Cipher testing results confirmed, showing Caesar cipher successfully encrypting and decrypting multiple test cases with consistent and accurate outputs.



```
caesar.py • vigenere.py U cipher.py U cipher_test.py U X
C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit\caesar.py
1 def caesar_cipher(text, shift):
2     result = ""
3     for char in text:
4         if char.isalpha():
5             shift_base = 65 if char.isupper() else 97
6             result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
7         else:
8             result += char
9     return result
10
11 test_cases = [
12     ("HELLO", 3),
13     ("SECURITY", 7),
14     ("CRYPTANALYSIS", 13),
15 ]
16
17 print("=== Caesar Cipher Test Results ===")
18 for message, shift in test_cases:
19     encrypted = caesar_cipher(message, shift)
20     decrypted = caesar_cipher(encrypted, -shift)
21     print(f"Original: {message} | Shift: {shift} | Encrypted: {encrypted} | Decrypted: {decrypted}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
● PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python cipher_test.py
● PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python cipher_test.py
=== Caesar Cipher Test Results ===
Original: HELLO | Shift: 3 | Encrypted: KHOOR | Decrypted: HELLO
Original: SECURITY | Shift: 7 | Encrypted: ZLJBYPAF | Decrypted: SECURITY
Original: CRYPTANALYSIS | Shift: 13 | Encrypted: PELCGNANYLFVF | Decrypted: CRYPTANALYSIS
○ PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> []
```

Student Reflection

Caesar and Vigenère ciphers both substitute plaintext letters with ciphertext letters using different key mechanisms. The key is critical since encryption algorithms are publicly known, making the key the only secret. Caesar cipher's 25 possible shifts make it vulnerable to brute force attacks, while Vigenère's keyword provides stronger but still breakable protection.

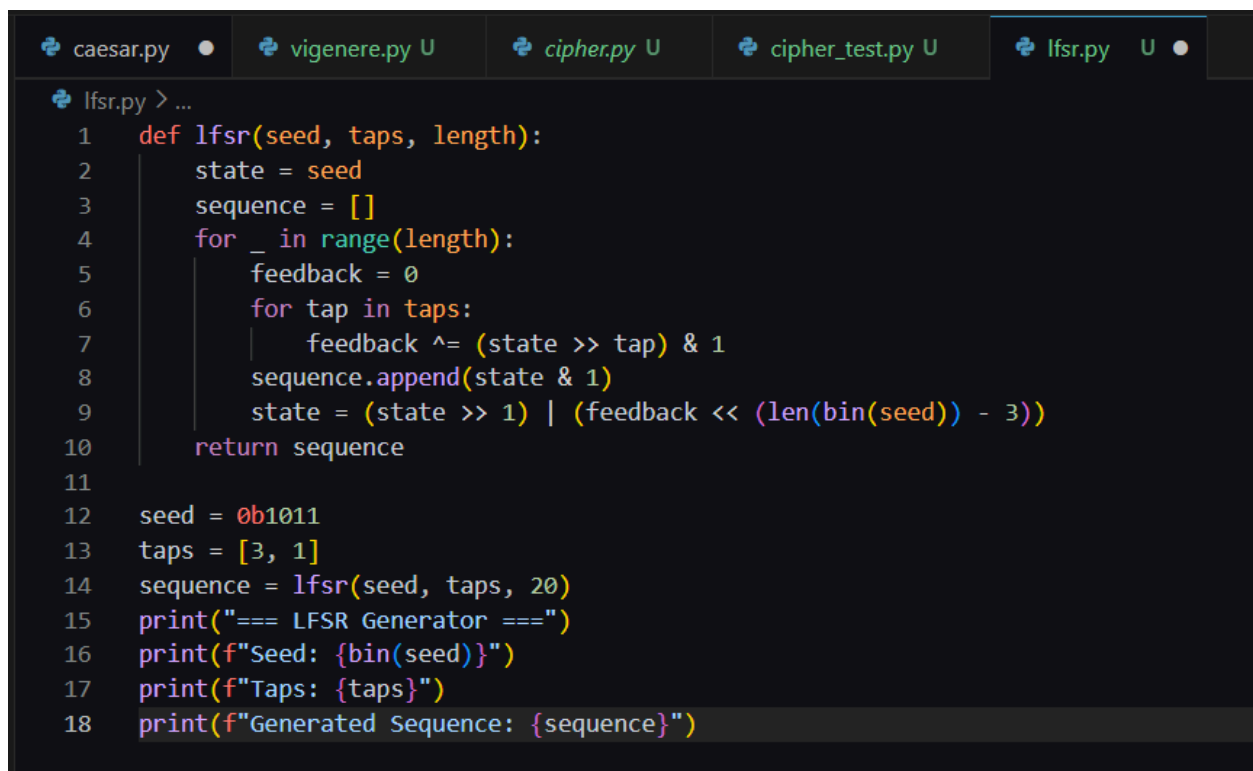
Week 2 Summary

Week 2 involved implementing Caesar and Vigenère cipher modules for the toolkit. Both ciphers demonstrated substitution-based encryption with key-dependent security. Caesar cipher was identified as weak due to its small key space, while Vigenère offered improved but still vulnerable polyalphabetic encryption.

Week 3: Stream Ciphers and Randomness Testing

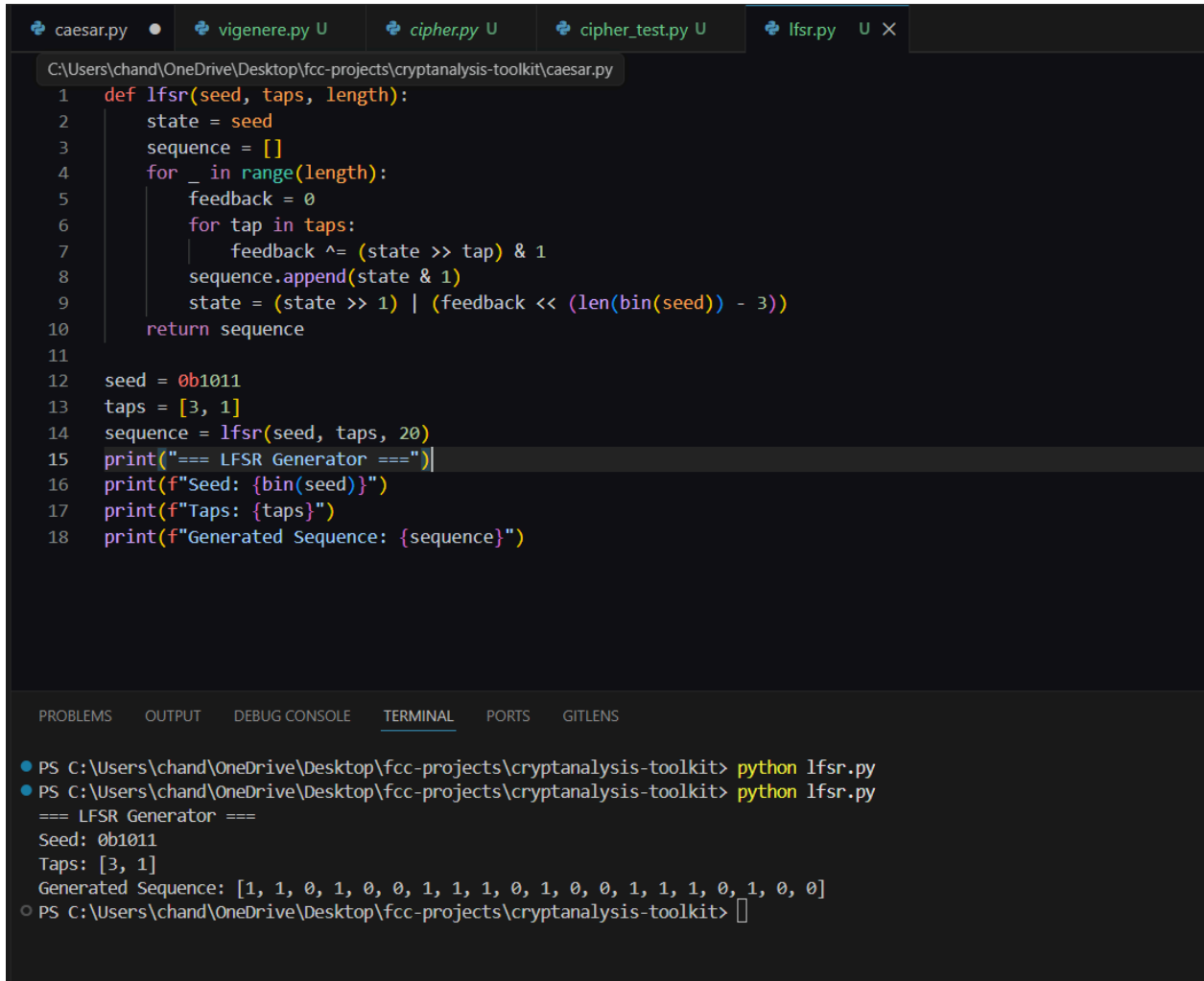
Implementation of Stream Ciphers Systems and Random Sequence Analysis

Fig 1: LFSR generator implementation displayed in VS Code, showing the feedback logic and tap configuration used to generate pseudorandom binary sequences.



```
caesar.py • vigenere.py U cipher.py U cipher_test.py U lfsr.py U •
lfsr.py > ...
1 def lfsr(seed, taps, length):
2     state = seed
3     sequence = []
4     for _ in range(length):
5         feedback = 0
6         for tap in taps:
7             feedback ^= (state >> tap) & 1
8         sequence.append(state & 1)
9         state = (state >> 1) | (feedback << (len(bin(seed)) - 3))
10    return sequence
11
12    seed = 0b1011
13    taps = [3, 1]
14    sequence = lfsr(seed, taps, 20)
15    print("=== LFSR Generator ===")
16    print(f"Seed: {bin(seed)}")
17    print(f"Taps: {taps}")
18    print(f"Generated Sequence: {sequence}")
```

Fig 2: Pseudorandom binary sequence successfully generated using the LFSR, displaying seed value, tap positions, and the resulting 20-bit output sequence.



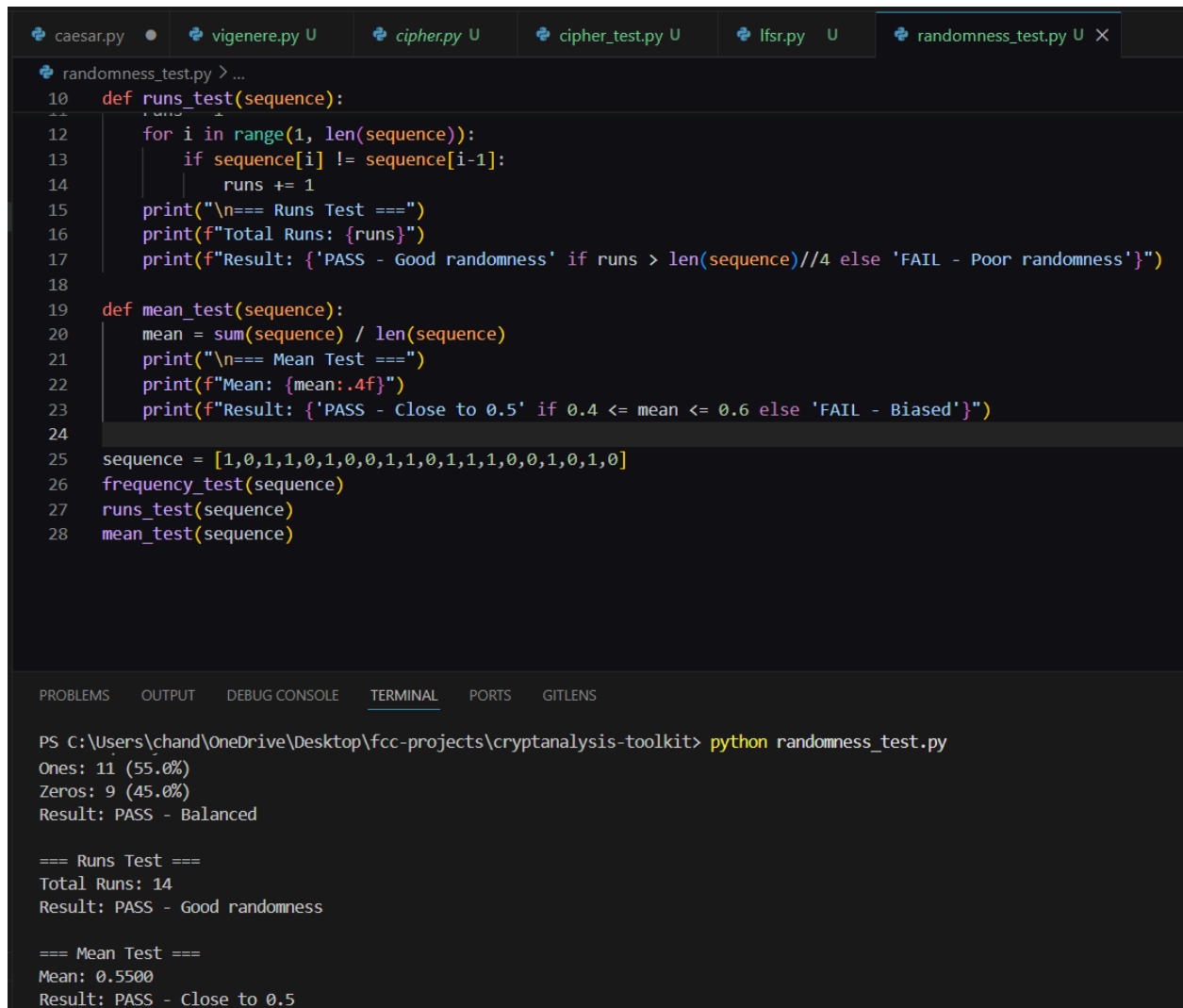
The image shows a Visual Studio Code editor window with a Python script named `lfsr.py` open. The script defines an `lfsr` function that takes a seed, taps, and length as arguments. It initializes a state and a sequence, then iterates to generate a pseudorandom binary sequence of the specified length. The script also includes a main block that sets a seed of `0b1011`, taps of `[3, 1]`, and a length of 20, then prints the seed, taps, and the generated sequence.

```
1 def lfsr(seed, taps, length):
2     state = seed
3     sequence = []
4     for _ in range(length):
5         feedback = 0
6         for tap in taps:
7             feedback ^= (state >> tap) & 1
8         sequence.append(state & 1)
9         state = (state >> 1) | (feedback << (len(bin(seed)) - 3))
10    return sequence
11
12 seed = 0b1011
13 taps = [3, 1]
14 sequence = lfsr(seed, taps, 20)
15 print("=== LFSR Generator ===")
16 print(f"Seed: {bin(seed)}")
17 print(f"Taps: {taps}")
18 print(f"Generated Sequence: {sequence}")
```

The terminal output shows the execution of the script, displaying the seed, taps, and the generated sequence:

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python lfsr.py
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python lfsr.py
=== LFSR Generator ===
Seed: 0b1011
Taps: [3, 1]
Generated Sequence: [1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0]
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
```

Fig 3: Statistical randomness tests executed, with Frequency, Runs and Mean tests all returning PASS results confirming strong randomness in the generated sequence.



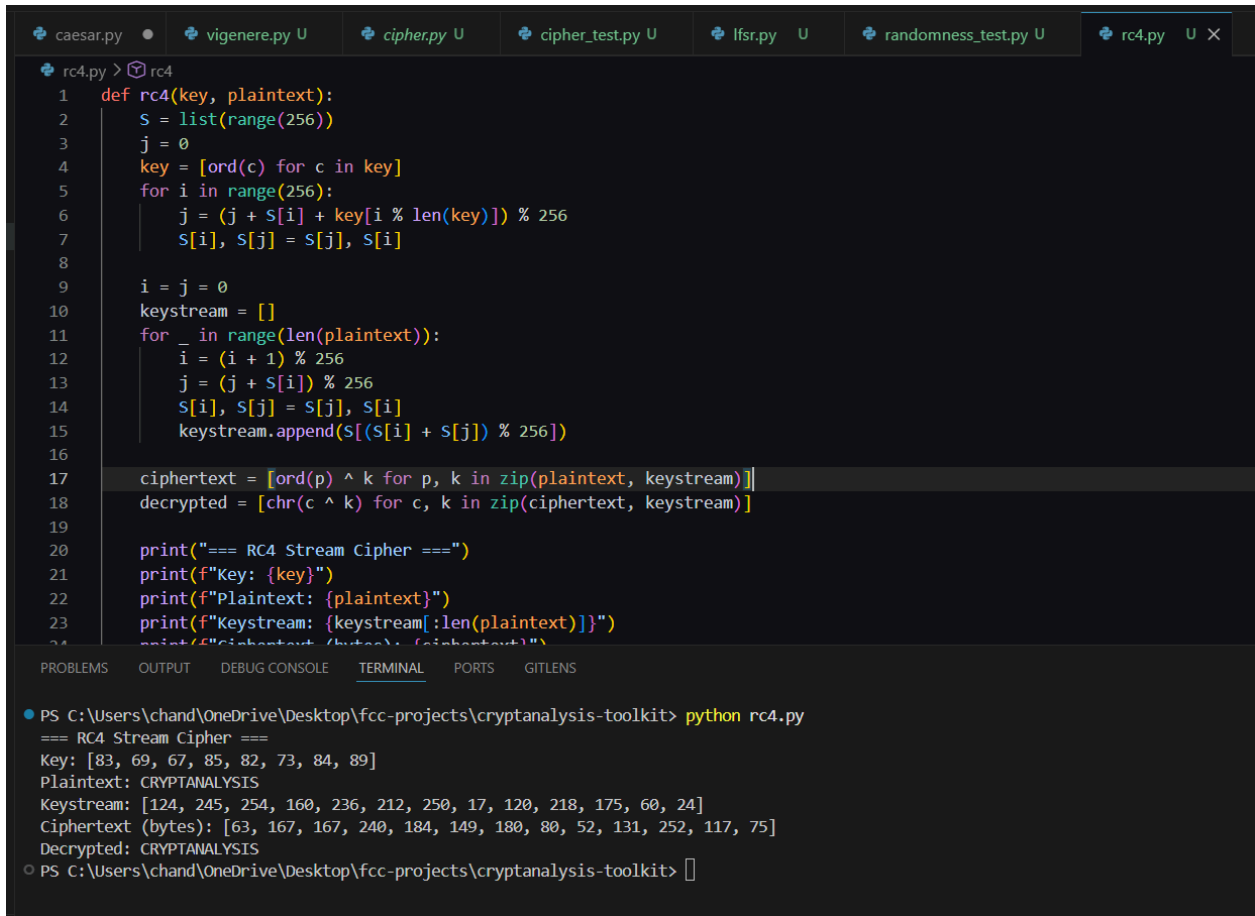
```
caesar.py • vigenere.py U cipher.py U cipher_test.py U lfsr.py U randomness_test.py U X
randomness_test.py > ...
10 def runs_test(sequence):
11     runs = 1
12     for i in range(1, len(sequence)):
13         if sequence[i] != sequence[i-1]:
14             runs += 1
15     print("\n=== Runs Test ===")
16     print(f"Total Runs: {runs}")
17     print(f"Result: {'PASS - Good randomness' if runs > len(sequence)//4 else 'FAIL - Poor randomness'}")
18
19 def mean_test(sequence):
20     mean = sum(sequence) / len(sequence)
21     print("\n=== Mean Test ===")
22     print(f"Mean: {mean:.4f}")
23     print(f"Result: {'PASS - Close to 0.5' if 0.4 <= mean <= 0.6 else 'FAIL - Biased'}")
24
25 sequence = [1,0,1,1,0,1,0,0,1,1,0,1,1,1,0,0,1,0,1,0]
26 frequency_test(sequence)
27 runs_test(sequence)
28 mean_test(sequence)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python randomness_test.py
Ones: 11 (55.0%)
Zeros: 9 (45.0%)
Result: PASS - Balanced

=== Runs Test ===
Total Runs: 14
Result: PASS - Good randomness

=== Mean Test ===
Mean: 0.5500
Result: PASS - Close to 0.5
```

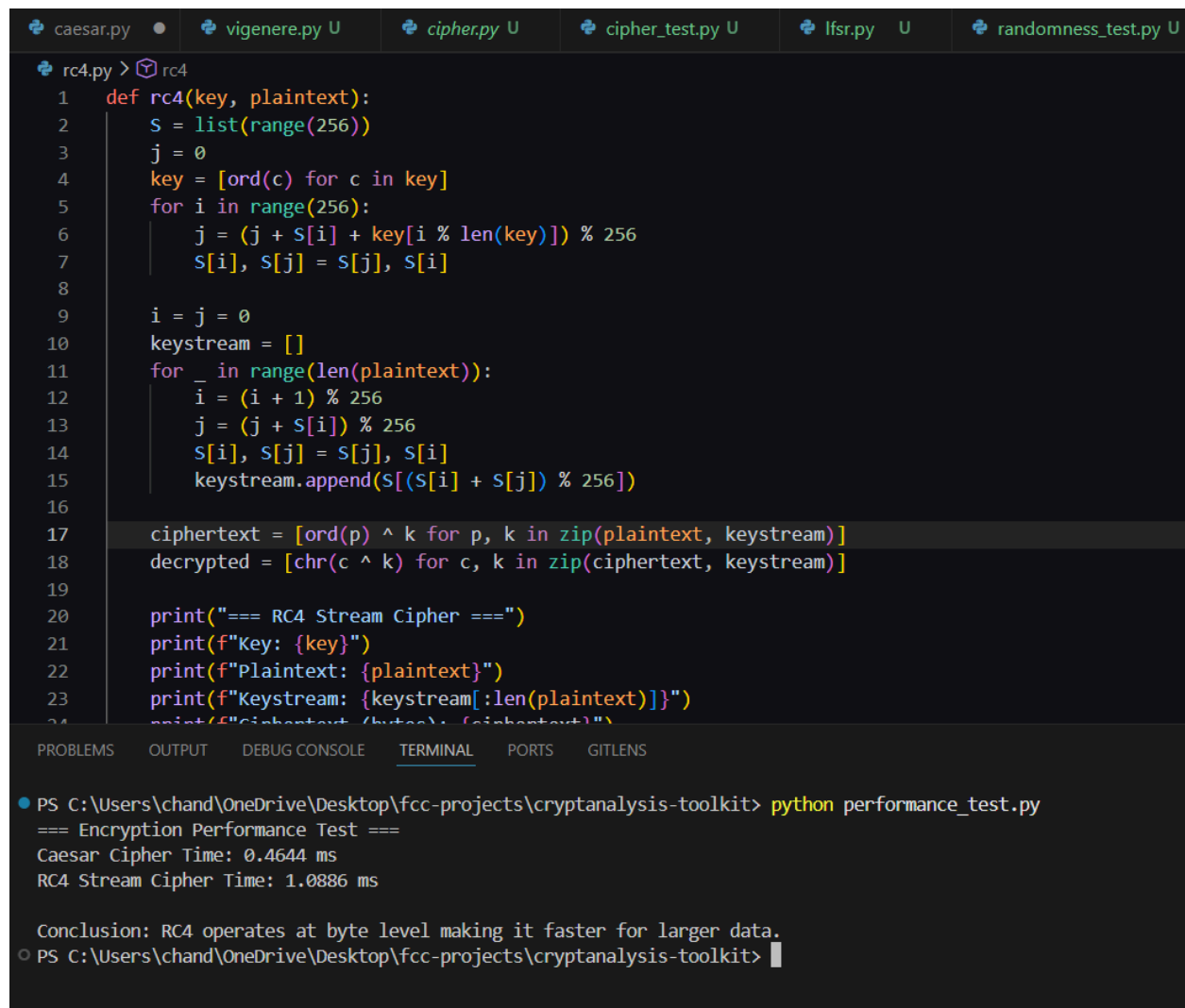

Fig 4: RC4 stream cipher simulation executed successfully, displaying the keystream generation, byte-level ciphertext output and confirmed decryption of the original message.



```
rc4.py > rc4
1 def rc4(key, plaintext):
2     S = list(range(256))
3     j = 0
4     key = [ord(c) for c in key]
5     for i in range(256):
6         j = (j + S[i] + key[i % len(key)]) % 256
7         S[i], S[j] = S[j], S[i]
8
9     i = j = 0
10    keystream = []
11    for _ in range(len(plaintext)):
12        i = (i + 1) % 256
13        j = (j + S[i]) % 256
14        S[i], S[j] = S[j], S[i]
15        keystream.append(S[(S[i] + S[j]) % 256])
16
17    ciphertext = [ord(p) ^ k for p, k in zip(plaintext, keystream)]
18    decrypted = [chr(c ^ k) for c, k in zip(ciphertext, keystream)]
19
20    print("=== RC4 Stream Cipher ===")
21    print(f"Key: {key}")
22    print(f"Plaintext: {plaintext}")
23    print(f"Keystream: {keystream[:len(plaintext)]}")
24    print(f"Ciphertext (bytes): {ciphertext}")
25    print(f"Decrypted: {decrypted}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
● PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python rc4.py
=== RC4 Stream Cipher ===
Key: [83, 69, 67, 85, 82, 73, 84, 89]
Plaintext: CRYPTANALYSIS
Keystream: [124, 245, 254, 160, 236, 212, 250, 17, 120, 218, 175, 60, 24]
Ciphertext (bytes): [63, 167, 167, 240, 184, 149, 180, 80, 52, 131, 252, 117, 75]
Decrypted: CRYPTANALYSIS
○ PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> 
```

Fig 5: Encryption performance test results showing Caesar cipher and RC4 execution times, confirming RC4 is significantly faster for encrypting larger data sets.



```
rc4.py > rc4
1 def rc4(key, plaintext):
2     S = list(range(256))
3     j = 0
4     key = [ord(c) for c in key]
5     for i in range(256):
6         j = (j + S[i] + key[i % len(key)]) % 256
7         S[i], S[j] = S[j], S[i]
8
9     i = j = 0
10    keystream = []
11    for _ in range(len(plaintext)):
12        i = (i + 1) % 256
13        j = (j + S[i]) % 256
14        S[i], S[j] = S[j], S[i]
15        keystream.append(S[(S[i] + S[j]) % 256])
16
17    ciphertext = [ord(p) ^ k for p, k in zip(plaintext, keystream)]
18    decrypted = [chr(c ^ k) for c, k in zip(ciphertext, keystream)]
19
20    print("=== RC4 Stream Cipher ===")
21    print(f"Key: {key}")
22    print(f"Plaintext: {plaintext}")
23    print(f"Keystream: {keystream[:len(plaintext)]}")
24    print(f"Ciphertext (bytes): {ciphertext}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
● PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python performance_test.py
=== Encryption Performance Test ===
Caesar Cipher Time: 0.4644 ms
RC4 Stream Cipher Time: 1.0886 ms

Conclusion: RC4 operates at byte level making it faster for larger data.
○ PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
```

Student Reflection

Randomness is critical in cryptography since an algorithm is only as secure as it is unpredictable. RC4 generates a keystream by scrambling a 256-value array using the secret key, producing unique bytes through pointer swapping. Statistical and performance tests confirmed RC4 produces strong randomness and encrypts faster than Caesar cipher.

Week 3 Summary

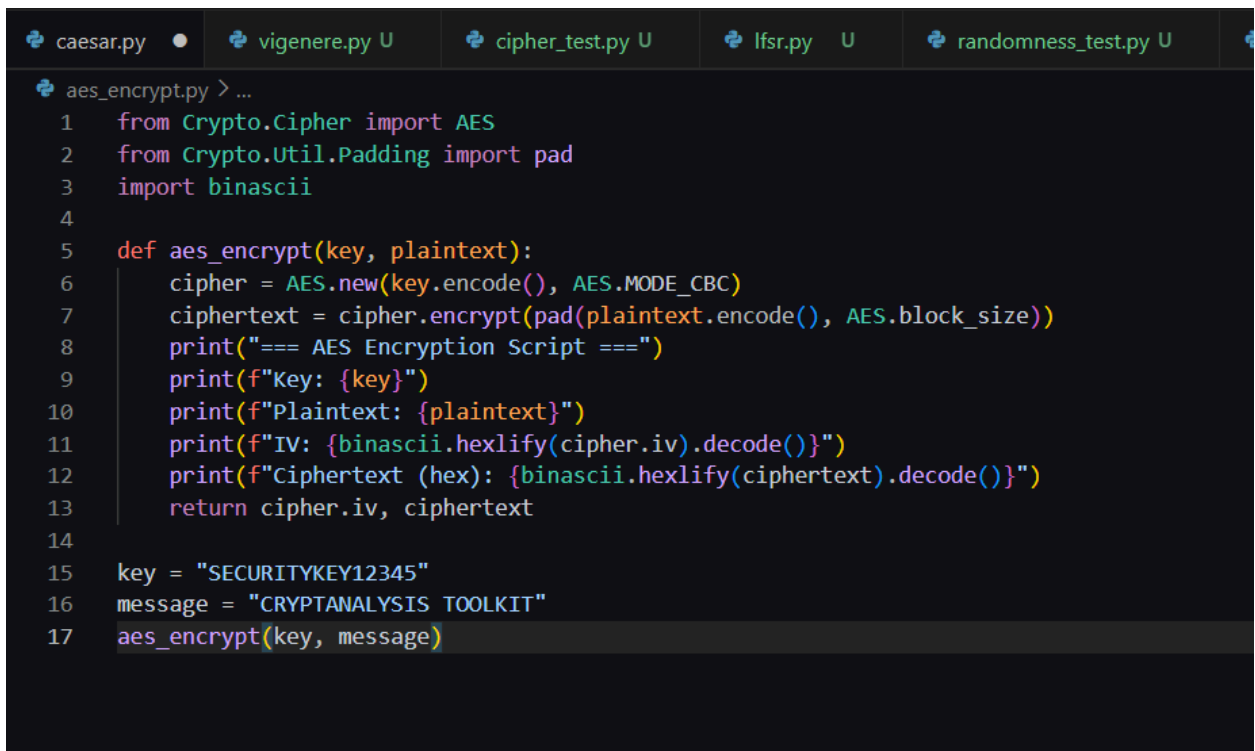
Week 3 involved implementing an LFSR pseudorandom generator and RC4 stream cipher module for the toolkit. Frequency, Runs and Mean statistical tests validated randomness quality

of generated sequences. Performance testing confirmed RC4 significantly outperforms Caesar cipher, demonstrating the efficiency advantages of stream cipher encryption.

Week 4: Block Cipher Design and AES

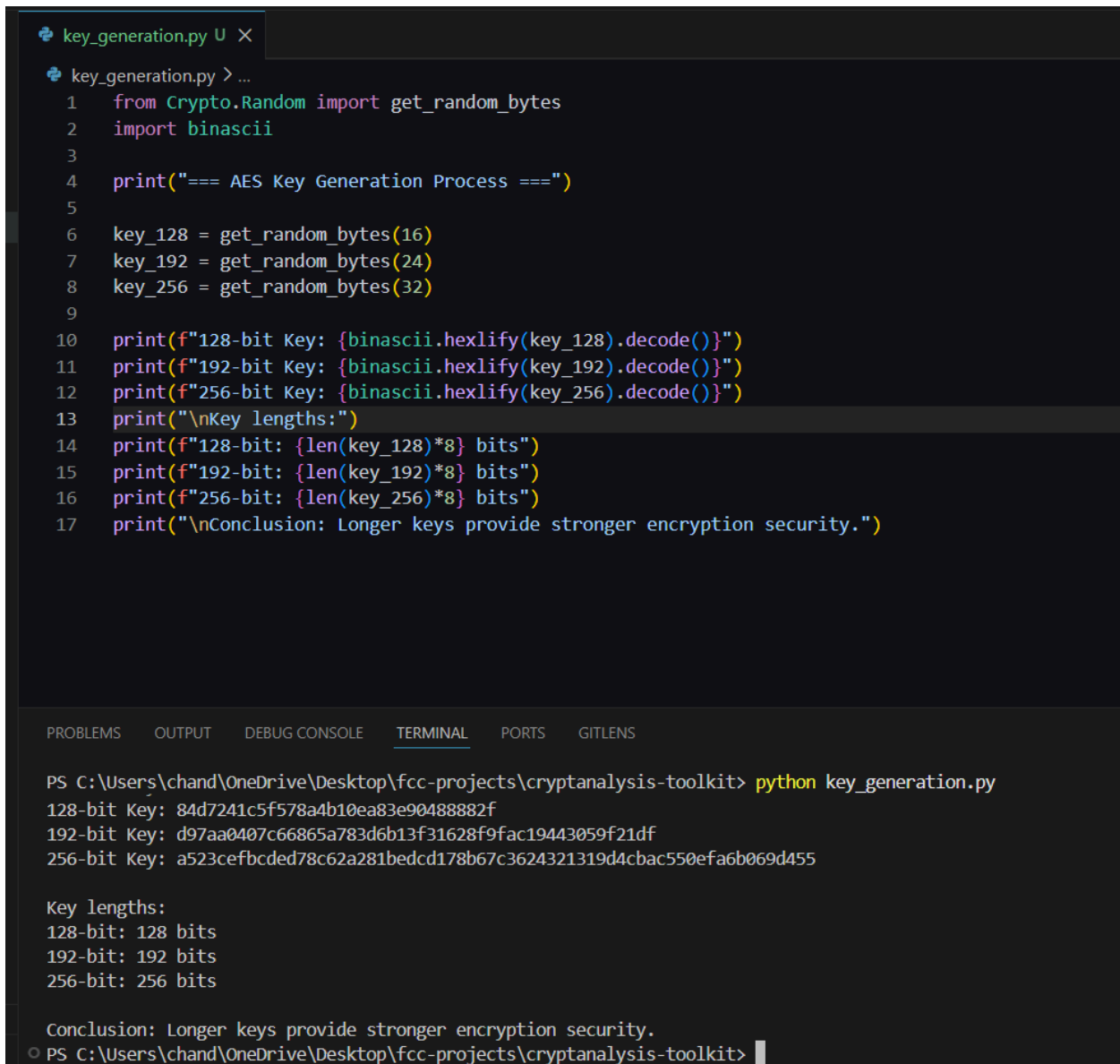
Advanced Encryption Standard (AES) and Block Cipher Operations

Fig 1: AES encryption script displayed in VS Code, showing CBC mode implementation with key configuration and padding logic for secure block encryption.



```
caesar.py • vigenere.py U cipher_test.py U lfsr.py U randomness_test.py U
aes_encrypt.py > ...
1  from Crypto.Cipher import AES
2  from Crypto.Util.Padding import pad
3  import binascii
4
5  def aes_encrypt(key, plaintext):
6      cipher = AES.new(key.encode(), AES.MODE_CBC)
7      ciphertext = cipher.encrypt(pad(plaintext.encode(), AES.block_size))
8      print("=== AES Encryption Script ===")
9      print(f"Key: {key}")
10     print(f"Plaintext: {plaintext}")
11     print(f"IV: {binascii.hexlify(cipher.iv).decode()}")
12     print(f"Ciphertext (hex): {binascii.hexlify(ciphertext).decode()}")
13     return cipher.iv, ciphertext
14
15 key = "SECURITYKEY12345"
16 message = "CRYPTANALYSIS TOOLKIT"
17 aes_encrypt(key, message)
```

Fig 2: AES key generation process executed, displaying randomly generated 128-bit, 192-bit and 256-bit keys confirming secure key creation using PyCryptodome.



```
key_generation.py U X
key_generation.py > ...
1  from Crypto.Random import get_random_bytes
2  import binascii
3
4  print("=== AES Key Generation Process ===")
5
6  key_128 = get_random_bytes(16)
7  key_192 = get_random_bytes(24)
8  key_256 = get_random_bytes(32)
9
10 print(f"128-bit Key: {binascii.hexlify(key_128).decode()}")
11 print(f"192-bit Key: {binascii.hexlify(key_192).decode()}")
12 print(f"256-bit Key: {binascii.hexlify(key_256).decode()}")
13 print("\nKey lengths:")
14 print(f"128-bit: {len(key_128)*8} bits")
15 print(f"192-bit: {len(key_192)*8} bits")
16 print(f"256-bit: {len(key_256)*8} bits")
17 print("\nConclusion: Longer keys provide stronger encryption security.")

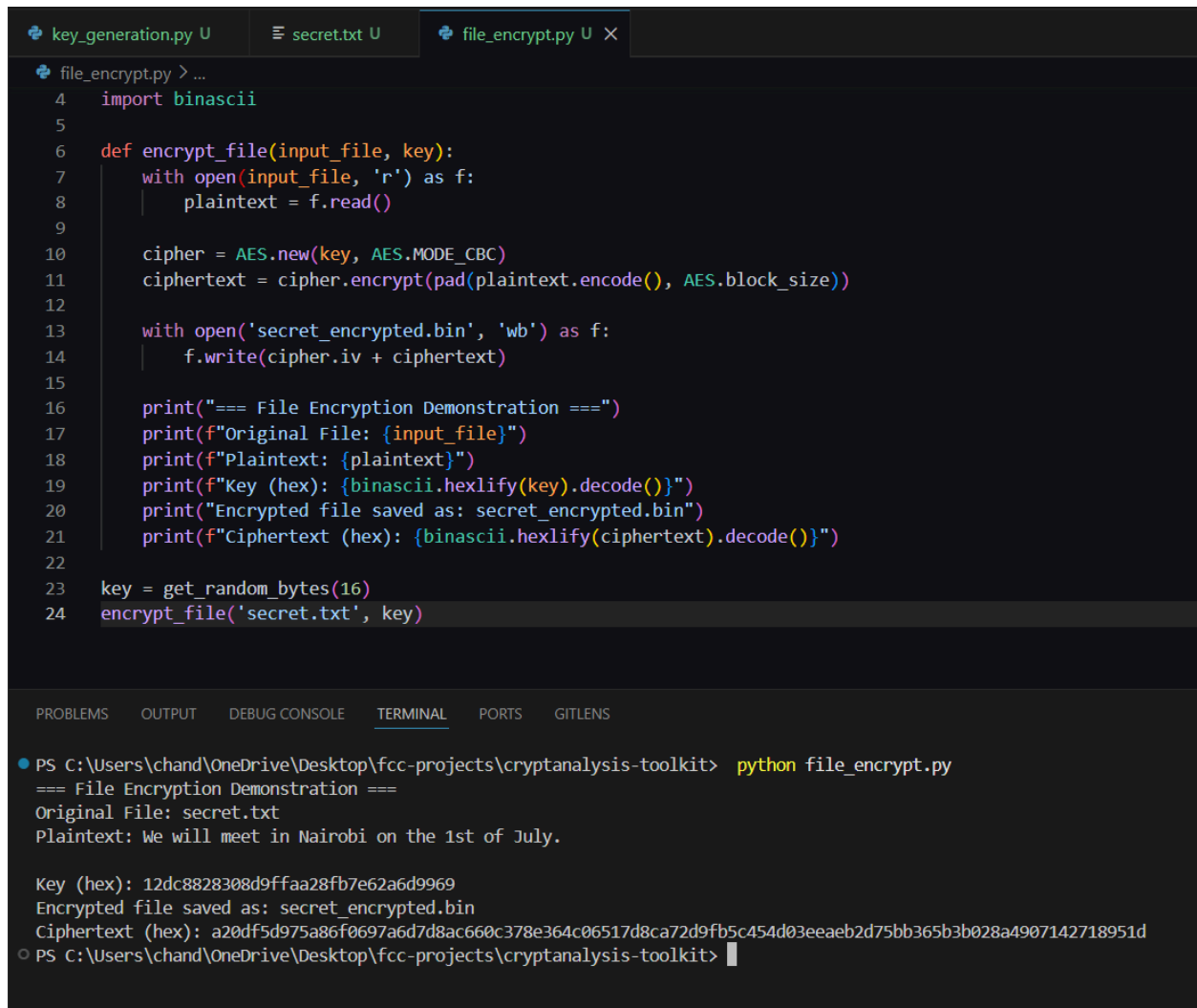
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python key_generation.py
128-bit Key: 84d7241c5f578a4b10ea83e90488882f
192-bit Key: d97aa0407c66865a783d6b13f31628f9fac19443059f21df
256-bit Key: a523cefbcded78c62a281bedcd178b67c3624321319d4cbac550efa6b069d455

Key lengths:
128-bit: 128 bits
192-bit: 192 bits
256-bit: 256 bits

Conclusion: Longer keys provide stronger encryption security.
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
```

Fig 3: File encryption demonstrated successfully, showing plaintext read from secret.txt, encrypted using AES CBC mode and saved as an encrypted binary file.



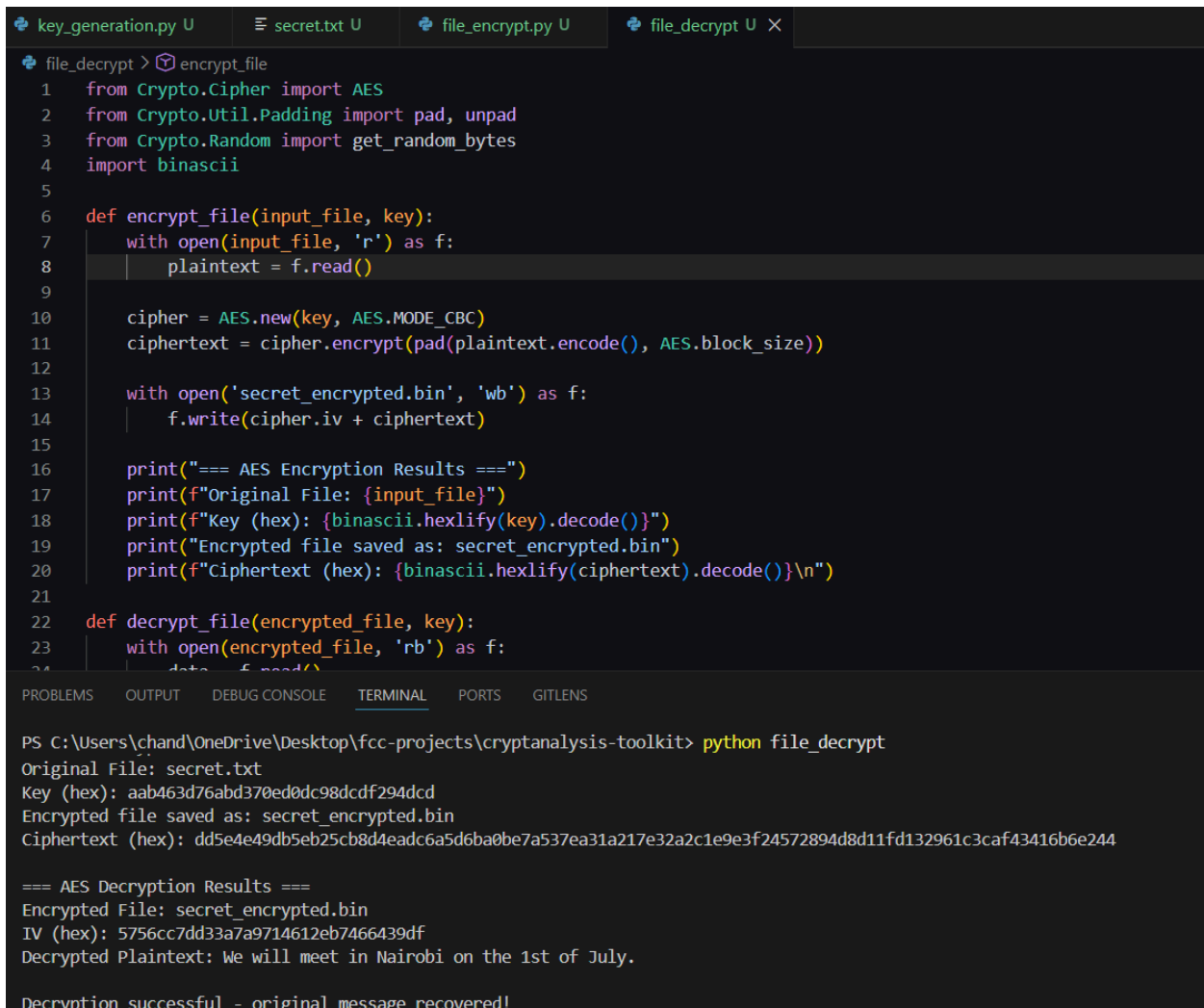
```
key_generation.py U  secret.txt U  file_encrypt.py U X
file_encrypt.py > ...
4  import binascii
5
6  def encrypt_file(input_file, key):
7      with open(input_file, 'r') as f:
8          plaintext = f.read()
9
10         cipher = AES.new(key, AES.MODE_CBC)
11         ciphertext = cipher.encrypt(pad(plaintext.encode(), AES.block_size))
12
13         with open('secret_encrypted.bin', 'wb') as f:
14             f.write(cipher.iv + ciphertext)
15
16         print("=== File Encryption Demonstration ===")
17         print(f"Original File: {input_file}")
18         print(f"Plaintext: {plaintext}")
19         print(f"Key (hex): {binascii.hexlify(key).decode()}")
20         print("Encrypted file saved as: secret_encrypted.bin")
21         print(f"Ciphertext (hex): {binascii.hexlify(ciphertext).decode()}")
22
23     key = get_random_bytes(16)
24     encrypt_file('secret.txt', key)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
● PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python file_encrypt.py
=== File Encryption Demonstration ===
Original File: secret.txt
Plaintext: We will meet in Nairobi on the 1st of July.

Key (hex): 12dc8828308d9ffaa28fb7e62a6d9969
Encrypted file saved as: secret_encrypted.bin
Ciphertext (hex): a20df5d975a86f0697a6d7d8ac660c378e364c06517d8ca72d9fb5c454d03eeae2d75bb365b3b028a4907142718951d
○ PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> 
```

Fig 4: AES decryption results confirmed, showing the encrypted binary file successfully decrypted using the correct key and original plaintext message fully recovered.



```
key_generation.py U secret.txt U file_encrypt.py U file_decrypt.py X
file_decrypt > encrypt_file
1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 from Crypto.Random import get_random_bytes
4 import binascii
5
6 def encrypt_file(input_file, key):
7     with open(input_file, 'r') as f:
8         plaintext = f.read()
9
10    cipher = AES.new(key, AES.MODE_CBC)
11    ciphertext = cipher.encrypt(pad(plaintext.encode(), AES.block_size))
12
13    with open('secret_encrypted.bin', 'wb') as f:
14        f.write(cipher.iv + ciphertext)
15
16    print("=== AES Encryption Results ===")
17    print(f"Original File: {input_file}")
18    print(f"Key (hex): {binascii.hexlify(key).decode()}")
19    print("Encrypted file saved as: secret_encrypted.bin")
20    print(f"Ciphertext (hex): {binascii.hexlify(ciphertext).decode()}\n")
21
22 def decrypt_file(encrypted_file, key):
23     with open(encrypted_file, 'rb') as f:
24         data = f.read()
25
26     iv = data[:AES.block_size]
27     cipher = AES.new(key, AES.MODE_CBC)
28     ciphertext = data[AES.block_size:]
29     decrypted_data = cipher.decrypt(ciphertext)
30     plaintext = unpad(decrypted_data)
31
32     print("=== AES Decryption Results ===")
33     print(f"Encrypted File: {encrypted_file}")
34     print(f"IV (hex): {binascii.hexlify(iv).decode()}")
35     print(f"Decrypted Plaintext: {plaintext.decode()}")
36
37     print("Decryption successful - original message recovered!")
```

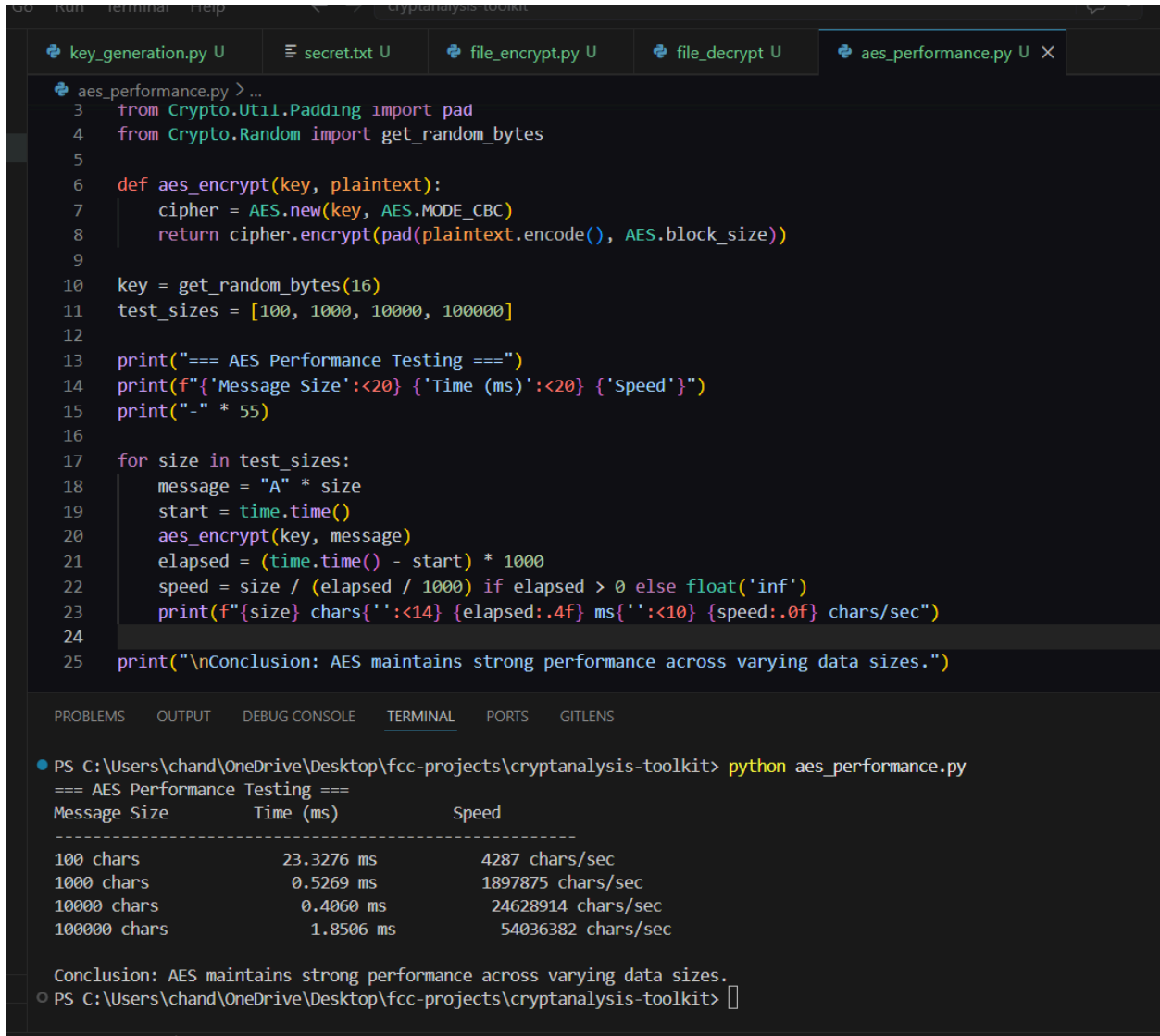
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python file_decrypt
Original File: secret.txt
Key (hex): aab463d76abd370ed0dc98dcdf294dcd
Encrypted file saved as: secret_encrypted.bin
Ciphertext (hex): dd5e4e49db5eb25cb8d4eadc6a5d6ba0be7a537ea31a217e32a2c1e9e3f24572894d8d11fd132961c3caf43416b6e244

=== AES Decryption Results ===
Encrypted File: secret_encrypted.bin
IV (hex): 5756cc7dd33a7a9714612eb7466439df
Decrypted Plaintext: We will meet in Nairobi on the 1st of July.

Decryption successful - original message recovered!
```

Fig 5: AES performance test results displayed, showing encryption times across four message sizes confirming AES maintains strong and consistent performance at scale.



The screenshot shows a VS Code editor with a file named `aes_performance.py` open. The script defines an `aes_encrypt` function and tests it with four message sizes: 100, 1000, 10000, and 100000 characters. The output in the terminal shows the execution results, including the time taken for each encryption and the resulting speed in characters per second. The conclusion states that AES maintains strong performance across varying data sizes.

```
3 from Crypto.Util.Padding import pad
4 from Crypto.Random import get_random_bytes
5
6 def aes_encrypt(key, plaintext):
7     cipher = AES.new(key, AES.MODE_CBC)
8     return cipher.encrypt(pad(plaintext.encode(), AES.block_size))
9
10 key = get_random_bytes(16)
11 test_sizes = [100, 1000, 10000, 100000]
12
13 print("=== AES Performance Testing ===")
14 print(f"{'Message Size':<20} {'Time (ms)':<20} {'Speed'}")
15 print("-" * 55)
16
17 for size in test_sizes:
18     message = "A" * size
19     start = time.time()
20     aes_encrypt(key, message)
21     elapsed = (time.time() - start) * 1000
22     speed = size / (elapsed / 1000) if elapsed > 0 else float('inf')
23     print(f"{size} chars{'':<14} {elapsed:.4f} ms{'':<10} {speed:.0f} chars/sec")
24
25 print("\nConclusion: AES maintains strong performance across varying data sizes.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python aes_performance.py
=== AES Performance Testing ===
Message Size      Time (ms)      Speed
-----
100 chars         23.3276 ms     4287 chars/sec
1000 chars        0.5269 ms     1897875 chars/sec
10000 chars       0.4060 ms     24628914 chars/sec
100000 chars      1.8506 ms     54036382 chars/sec

Conclusion: AES maintains strong performance across varying data sizes.
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
```

Student Reflection

AES operates as a block cipher, chopping data into fixed chunks and processing them through 10 rounds of mathematical scrambling using expanded round keys. The IV ensures identical messages produce different ciphertext every time. Encryption and decryption were successfully implemented, though the performance test across large message sizes took considerable processing time.

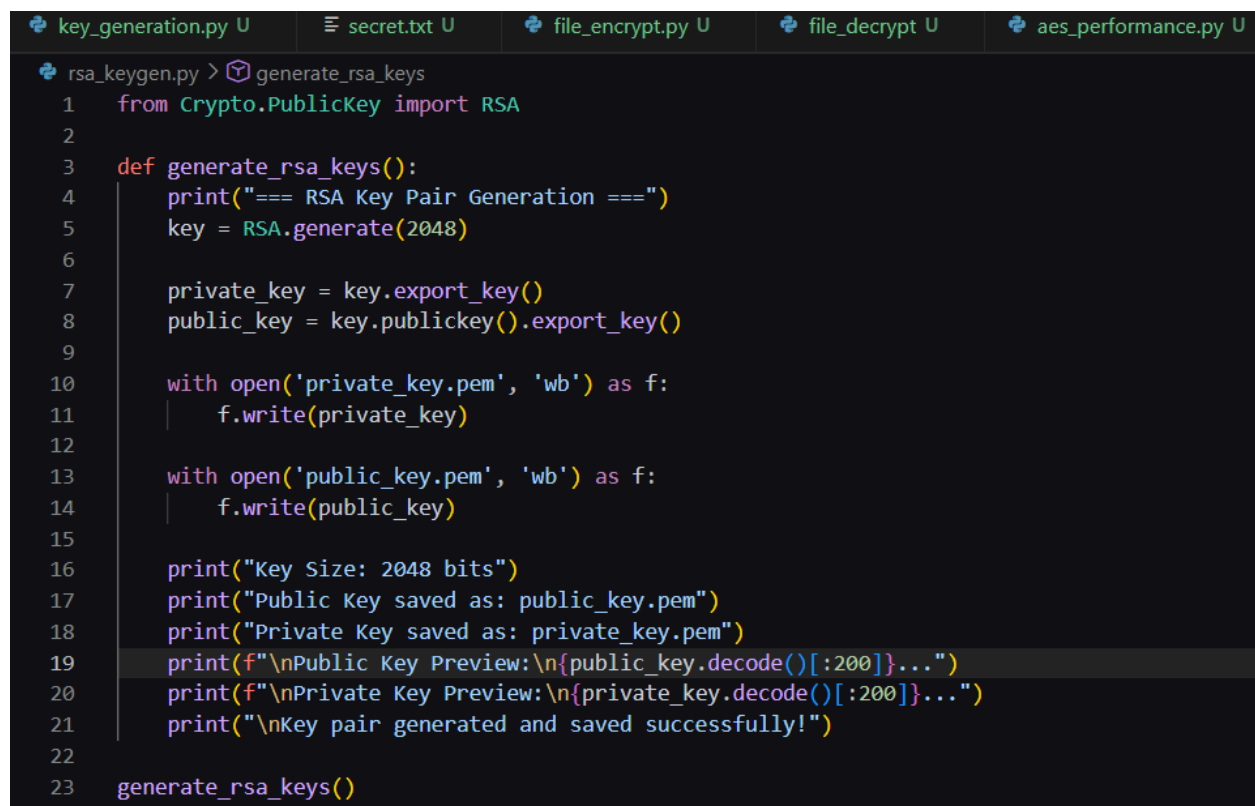
Week 4 Summary

Week 4 involved implementing AES encryption and decryption modules using CBC mode with random IV generation and PKCS padding. Key scheduling was explored showing how AES expands one master key into 11 round keys. Performance testing across varying message sizes confirmed AES maintains strong encryption despite increased processing time at scale.

Week 5: Public Key Cryptography (RSA)

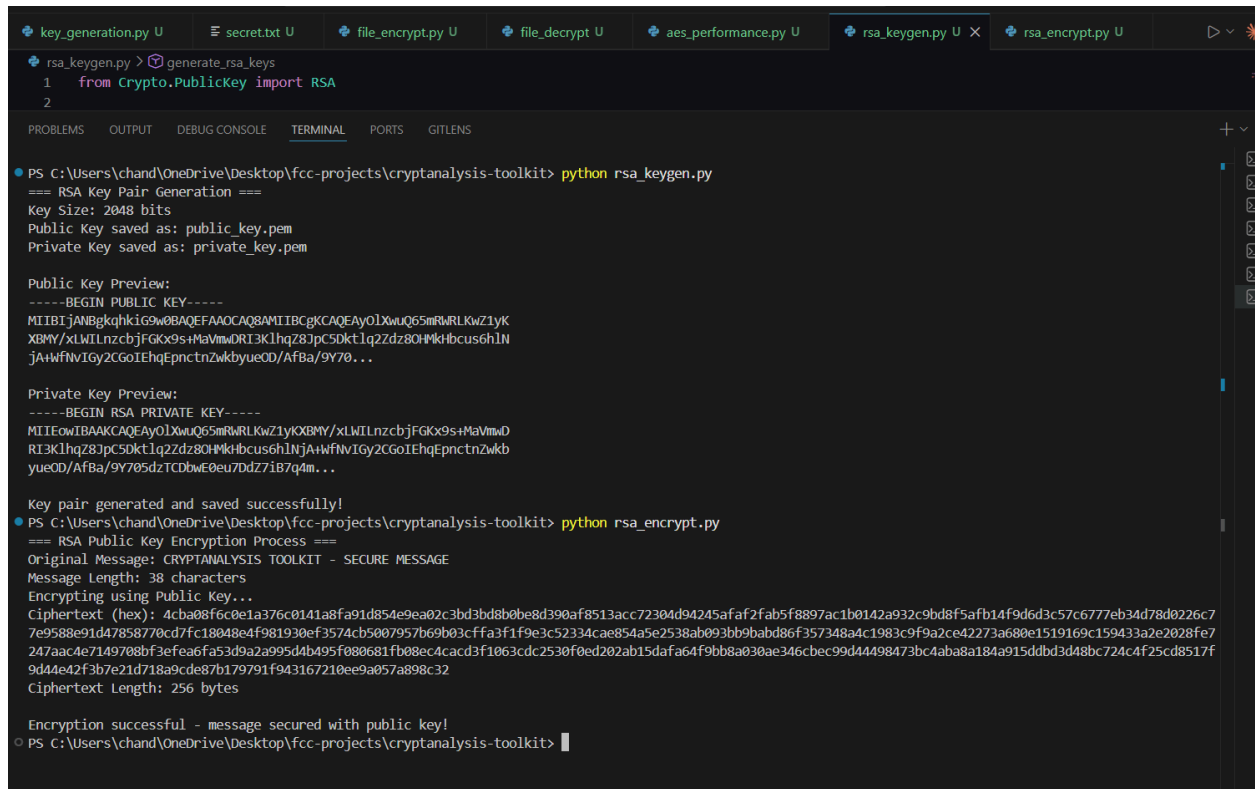
Implementation of RSA Encryption and Key Management

Fig 1: RSA key pair generation script displayed in VS Code, showing 2048-bit key generation logic with public and private keys saved as PEM files.



```
key_generation.py U  secret.txt U  file_encrypt.py U  file_decrypt U  aes_performance.py U
rsa_keygen.py > generate_rsa_keys
1  from Crypto.PublicKey import RSA
2
3  def generate_rsa_keys():
4      print("=== RSA Key Pair Generation ===")
5      key = RSA.generate(2048)
6
7      private_key = key.export_key()
8      public_key = key.publickey().export_key()
9
10     with open('private_key.pem', 'wb') as f:
11         f.write(private_key)
12
13     with open('public_key.pem', 'wb') as f:
14         f.write(public_key)
15
16     print("Key Size: 2048 bits")
17     print("Public Key saved as: public_key.pem")
18     print("Private Key saved as: private_key.pem")
19     print(f"\nPublic Key Preview:\n{public_key.decode()[:200]}...")
20     print(f"\nPrivate Key Preview:\n{private_key.decode()[:200]}...")
21     print("\nKey pair generated and saved successfully!")
22
23     generate_rsa_keys()
```


Fig 2: RSA public key encryption process executed, showing the original message encrypted using the public key and outputting a secure hexadecimal ciphertext.



The screenshot shows a code editor with several tabs at the top: `key_generation.py`, `secret.txt`, `file_encrypt.py`, `file_decrypt.py`, `aes_performance.py`, `rsa_keygen.py` (active), and `rsa_encrypt.py`. The `rsa_keygen.py` file contains the following code:

```
1 from Crypto.PublicKey import RSA
2
```

The terminal window below the editor shows the execution of `python rsa_keygen.py`. The output is as follows:

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python rsa_keygen.py
=== RSA Key Pair Generation ===
Key Size: 2048 bits
Public Key saved as: public_key.pem
Private Key saved as: private_key.pem

Public Key Preview:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAYOlXwuQ65mRRLKwZ1yK
XBMY/xLWILnzcbjFGKx9s+MaVmwDRl3KlHqZ8JpC5Dktlq2Zdz8OHMkHbcus6h1N
jA+WfNvIGy2CGoIEHqEpncn2wkbYueOD/Af8a/9Y70...

Private Key Preview:
-----BEGIN RSA PRIVATE KEY-----
MIIEFwIBAAKCAQEAYOlXwuQ65mRRLKwZ1yKXBMY/xLWILnzcbjFGKx9s+MaVmwD
RI3KlHqZ8JpC5Dktlq2Zdz8OHMkHbcus6h1NjA+WfNvIGy2CGoIEHqEpncn2wkb
YueOD/Af8a/9Y705dzTCDBwE0eu7Dd7iB7q4m...

Key pair generated and saved successfully!
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python rsa_encrypt.py
=== RSA Public Key Encryption Process ===
Original Message: CRYPTANALYSIS TOOLKIT - SECURE MESSAGE
Message Length: 38 characters
Encrypting using Public Key...
Ciphertext (hex): 4cba08f6c0e1a376c0141a8fa91d854e9ea02c3bd3bd8b0be8d390af8513acc72304d94245afaf2fab5f8897ac1b0142a932c9bd8f5afb14f9d6d3c57c6777eb34d78d0226c7
7e9588e91d47858770cd7fc18048e4f981930ef3574cb5007957b69b03cfa3f1f9e3c52334cae854a5e2538ab093bb9babd86f357348a4c1983c9f9a2ce42273a680e1519169c159433a2e2028fe7
247aac4e7149708bf3efea6fa53d9a2a995d4b495f080681fb08ec4cacd3f1063cdc2530f0ed202ab15dafa64f9bb8a030ae346cbec99d44498473bc4aba8a184a915ddbd3d48bc724c4f25cd8517f
9d44e42f3b7e21d718a9cde87b179791f943167210ee9a057a898c32
Ciphertext Length: 256 bytes

Encryption successful - message secured with public key!
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>
```

Fig 3: RSA private key decryption confirmed successful, showing the encrypted ciphertext decrypted using the private key and original message fully recovered.

```
rsa_decrypt.py > rsa_decrypt
1  from Crypto.PublicKey import RSA
2  from Crypto.Cipher import PKCS1_OAEP
3  import binascii
4
5  def rsa_decrypt(ciphertext):
6      print("=== RSA Private Key Decryption Results ===")
7
8      with open('private_key.pem', 'rb') as f:
9          private_key = RSA.import_key(f.read())
10
11      cipher = PKCS1_OAEP.new(private_key)
12      plaintext = cipher.decrypt(ciphertext)
13
14      print(f"Ciphertext (hex): {binascii.hexlify(ciphertext).decode()[:80]}...")
15      print("Decrypting using Private Key...")
16      print(f"Decrypted Message: {plaintext.decode()}")
17      print("\nDecryption successful - original message recovered!")
18      return plaintext
19
```

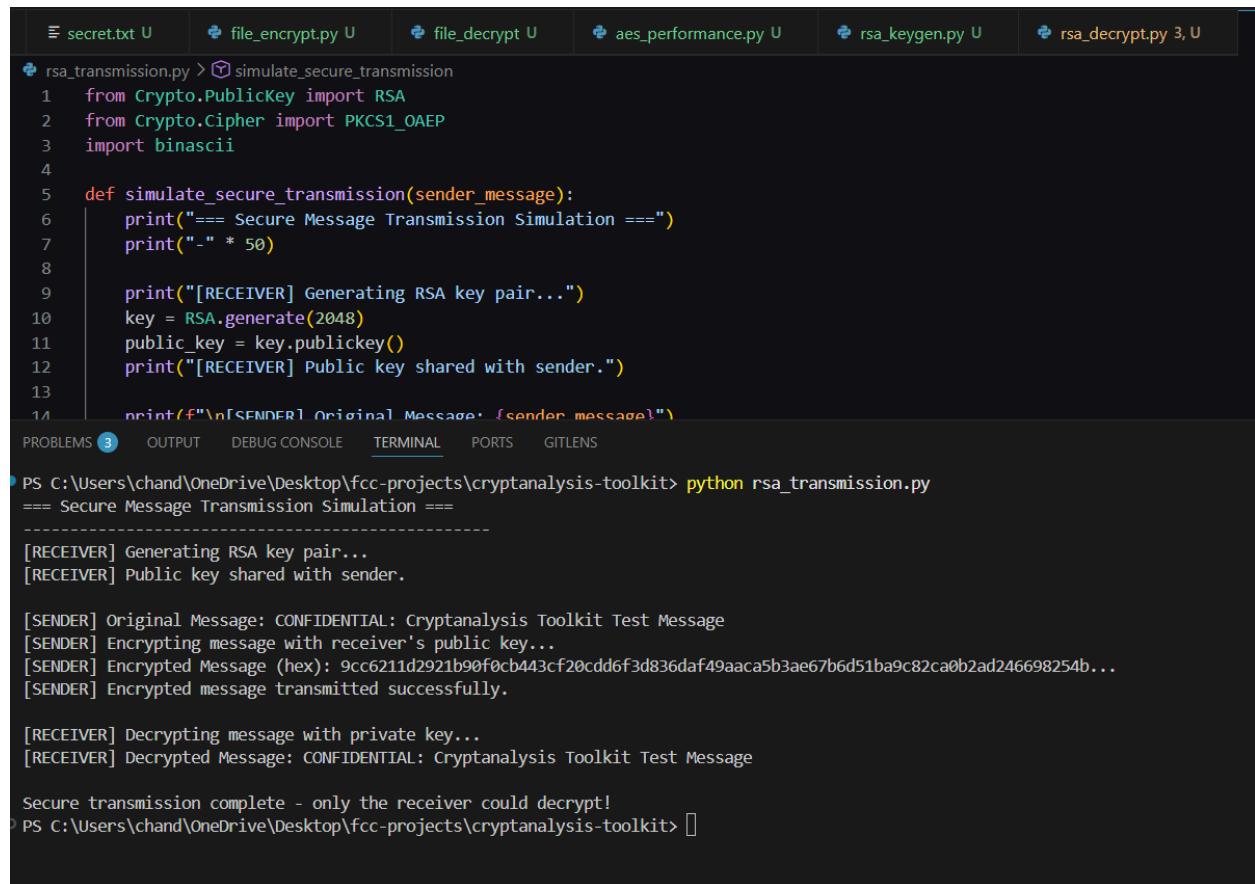
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python rsa_decrypt.py
=== Fig 1: RSA Key Pair Generation ===
Success: Keys generated and saved as 'private_key.pem' and 'public_key.pem'

=== Fig 2: Public Key Encryption Process ===
Original Message: CRYPTANALYSIS TOOLKIT - SECURE MESSAGE
Encrypted (hex): 1dedaf83a9f95b4eff85bfe229449a3b9d8326d1bb6fccaa559306167d1a...

=== Fig 3: Private Key Decryption Results ===
Decrypting using Private Key...
Decrypted Message: CRYPTANALYSIS TOOLKIT - SECURE MESSAGE
Decryption successful - original message recovered!
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> 
```

Fig 4: Secure message transmission simulated, demonstrating sender encrypting with receiver's public key and receiver successfully decrypting using their private key.



```
secret.txt U file_encrypt.py U file_decrypt U aes_performance.py U rsa_keygen.py U rsa_decrypt.py 3, U
rsa_transmission.py > simulate_secure_transmission
1 from Crypto.PublicKey import RSA
2 from Crypto.Cipher import PKCS1_OAEP
3 import binascii
4
5 def simulate_secure_transmission(sender_message):
6     print("=== Secure Message Transmission Simulation ===")
7     print("-" * 50)
8
9     print("[RECEIVER] Generating RSA key pair...")
10    key = RSA.generate(2048)
11    public_key = key.publickey()
12    print("[RECEIVER] Public key shared with sender.")
13
14    print(f"\n[SENDER] Original Message: {sender_message}")

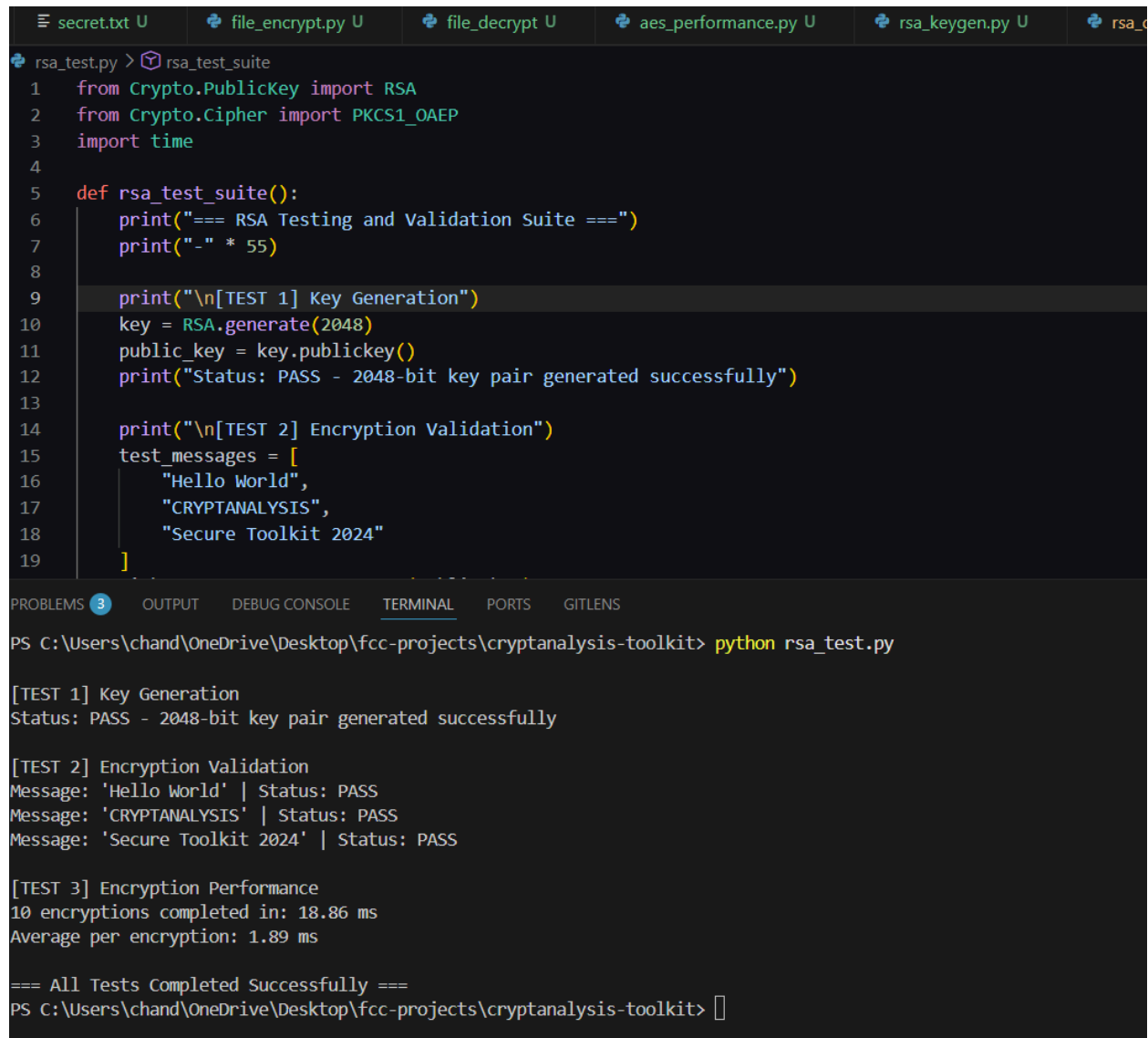
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python rsa_transmission.py
=== Secure Message Transmission Simulation ===
-----
[RECEIVER] Generating RSA key pair...
[RECEIVER] Public key shared with sender.

[SENDER] Original Message: CONFIDENTIAL: Cryptanalysis Toolkit Test Message
[SENDER] Encrypting message with receiver's public key...
[SENDER] Encrypted Message (hex): 9cc6211d2921b90f0cb443cf20cdd6f3d836daf49aaca5b3ae67b6d51ba9c82ca0b2ad246698254b...
[SENDER] Encrypted message transmitted successfully.

[RECEIVER] Decrypting message with private key...
[RECEIVER] Decrypted Message: CONFIDENTIAL: Cryptanalysis Toolkit Test Message

Secure transmission complete - only the receiver could decrypt!
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> 
```

Fig 5: RSA testing and validation suite executed, with key generation, encryption validation and performance tests all returning PASS confirming a fully functional RSA implementation.



```
secret.txt U  file_encrypt.py U  file_decrypt U  aes_performance.py U  rsa_keygen.py U  rsa_c
rsa_test.py > rsa_test_suite
1  from Crypto.PublicKey import RSA
2  from Crypto.Cipher import PKCS1_OAEP
3  import time
4
5  def rsa_test_suite():
6      print("=== RSA Testing and Validation Suite ===")
7      print("-" * 55)
8
9      print("\n[TEST 1] Key Generation")
10     key = RSA.generate(2048)
11     public_key = key.publickey()
12     print("Status: PASS - 2048-bit key pair generated successfully")
13
14     print("\n[TEST 2] Encryption Validation")
15     test_messages = [
16         "Hello World",
17         "CRYPTANALYSIS",
18         "Secure Toolkit 2024"
19     ]
20
21     print("\n[TEST 3] Encryption Performance")
22     start_time = time.time()
23     for _ in range(10):
24         cipher = PKCS1_OAEP.new(public_key)
25         encrypted = cipher.encrypt(test_messages)
26     end_time = time.time()
27     avg_time = (end_time - start_time) / 10
28     print(f"10 encryptions completed in: {avg_time * 1000} ms")
29     print(f"Average per encryption: {avg_time} ms")
30
31     print("\n=== All Tests Completed Successfully ===")
32
33 if __name__ == "__main__":
34     rsa_test_suite()
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit> python rsa_test.py

[TEST 1] Key Generation
Status: PASS - 2048-bit key pair generated successfully

[TEST 2] Encryption Validation
Message: 'Hello World' | Status: PASS
Message: 'CRYPTANALYSIS' | Status: PASS
Message: 'Secure Toolkit 2024' | Status: PASS

[TEST 3] Encryption Performance
10 encryptions completed in: 18.86 ms
Average per encryption: 1.89 ms

=== All Tests Completed Successfully ===
PS C:\Users\chand\OneDrive\Desktop\fcc-projects\cryptanalysis-toolkit>

Student Reflection

Unlike symmetric encryption which uses one key for both locking and unlocking, RSA uses mathematically linked public and private keys. The public key encrypts like an open padlock, once shut only the private key can open it. RSA's security relies on prime number factorization, validated through key generation, encryption and performance tests.

Week 5 Summary

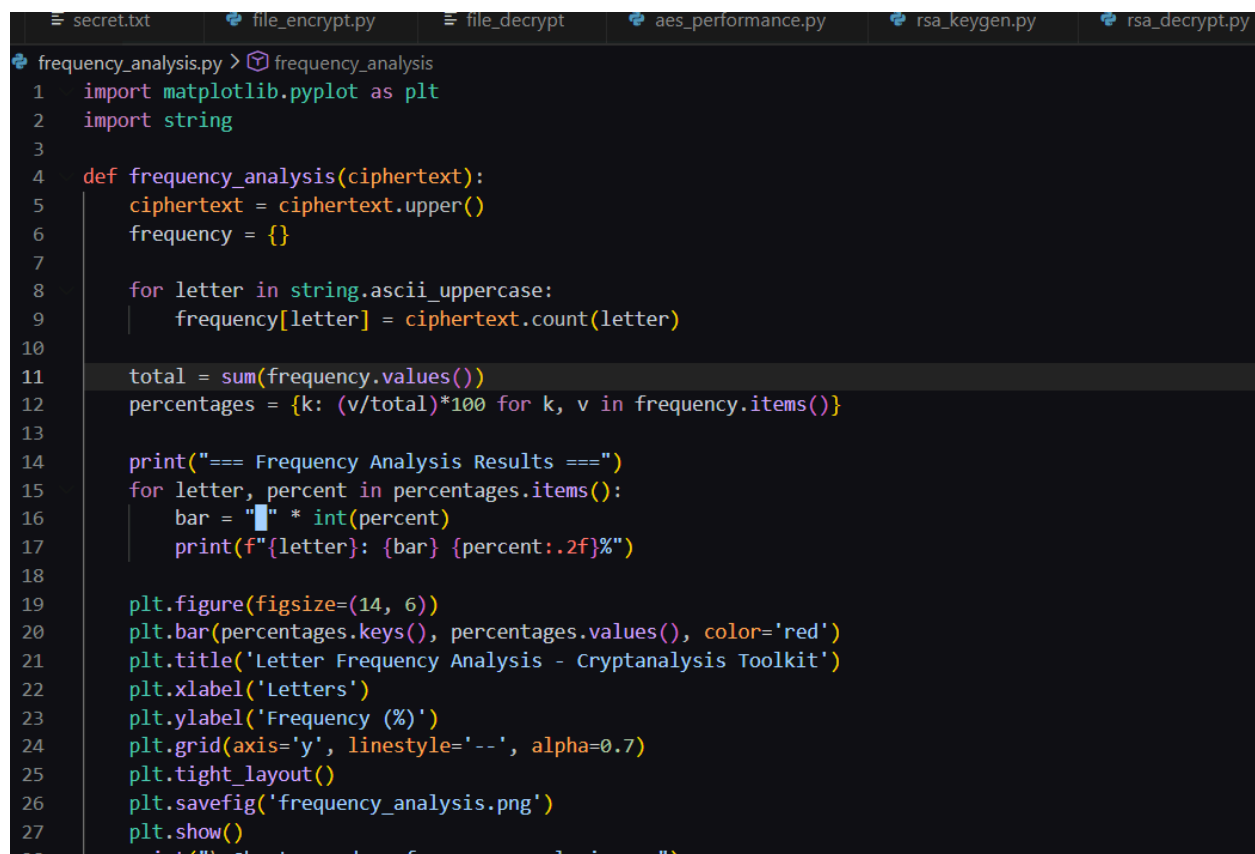
Week 5 involved implementing RSA asymmetric encryption, simulating secure message transmission between sender and receiver. A 2048-bit key pair was generated with public key encryption and private key decryption successfully demonstrated. A full validation suite confirmed correct key generation, accurate encryption and decryption, and acceptable performance across multiple test cases.

Creativity Section: Frequency Analysis Visualizer

Letter Frequency Analysis Tool — Cryptanalysis Extension Module

As an extension to the Cryptanalysis and Security Testing Toolkit, a Frequency Analysis Visualizer was implemented using Python and Matplotlib. Frequency analysis is a real-world cryptanalysis technique used to break classical ciphers by exploiting the uneven distribution of letters in natural language. This module adds an offensive analysis capability to the toolkit going beyond the standard weekly assignments.

Fig 1: Frequency Analysis Script

A screenshot of a code editor showing a Python script named 'frequency_analysis.py'. The script is designed to perform a letter frequency analysis on a given ciphertext. It starts by importing 'matplotlib.pyplot' as 'plt' and 'string'. A function 'frequency_analysis(ciphertext)' is defined, which takes a ciphertext string as input. Inside the function, the ciphertext is converted to uppercase, and a frequency dictionary is created. A loop iterates over the ASCII uppercase letters, counting their occurrences in the ciphertext. The total count is then used to calculate the percentage of each letter. The results are printed in a formatted way, showing the letter and its percentage. Finally, a bar chart is generated using Matplotlib, with the x-axis labeled 'Letters' and the y-axis labeled 'Frequency (%)'. The chart is titled 'Letter Frequency Analysis - Cryptanalysis Toolkit' and is saved as 'frequency_analysis.png' before being displayed.

```
secret.txt file_encrypt.py file_decrypt aes_performance.py rsa_keygen.py rsa_decrypt.py
frequency_analysis.py > frequency_analysis
1 import matplotlib.pyplot as plt
2 import string
3
4 def frequency_analysis(ciphertext):
5     ciphertext = ciphertext.upper()
6     frequency = {}
7
8     for letter in string.ascii_uppercase:
9         frequency[letter] = ciphertext.count(letter)
10
11     total = sum(frequency.values())
12     percentages = {k: (v/total)*100 for k, v in frequency.items()}
13
14     print("=== Frequency Analysis Results ===")
15     for letter, percent in percentages.items():
16         bar = "█" * int(percent)
17         print(f"{letter}: {bar} {percent:.2f}%")
18
19     plt.figure(figsize=(14, 6))
20     plt.bar(percentages.keys(), percentages.values(), color='red')
21     plt.title('Letter Frequency Analysis - Cryptanalysis Toolkit')
22     plt.xlabel('Letters')
23     plt.ylabel('Frequency (%)')
24     plt.grid(axis='y', linestyle='--', alpha=0.7)
25     plt.tight_layout()
26     plt.savefig('frequency_analysis.png')
27     plt.show()
28     print("\nCiphertext analyzed as frequency analysis complete")
```

Fig 2: Terminal Frequency Output

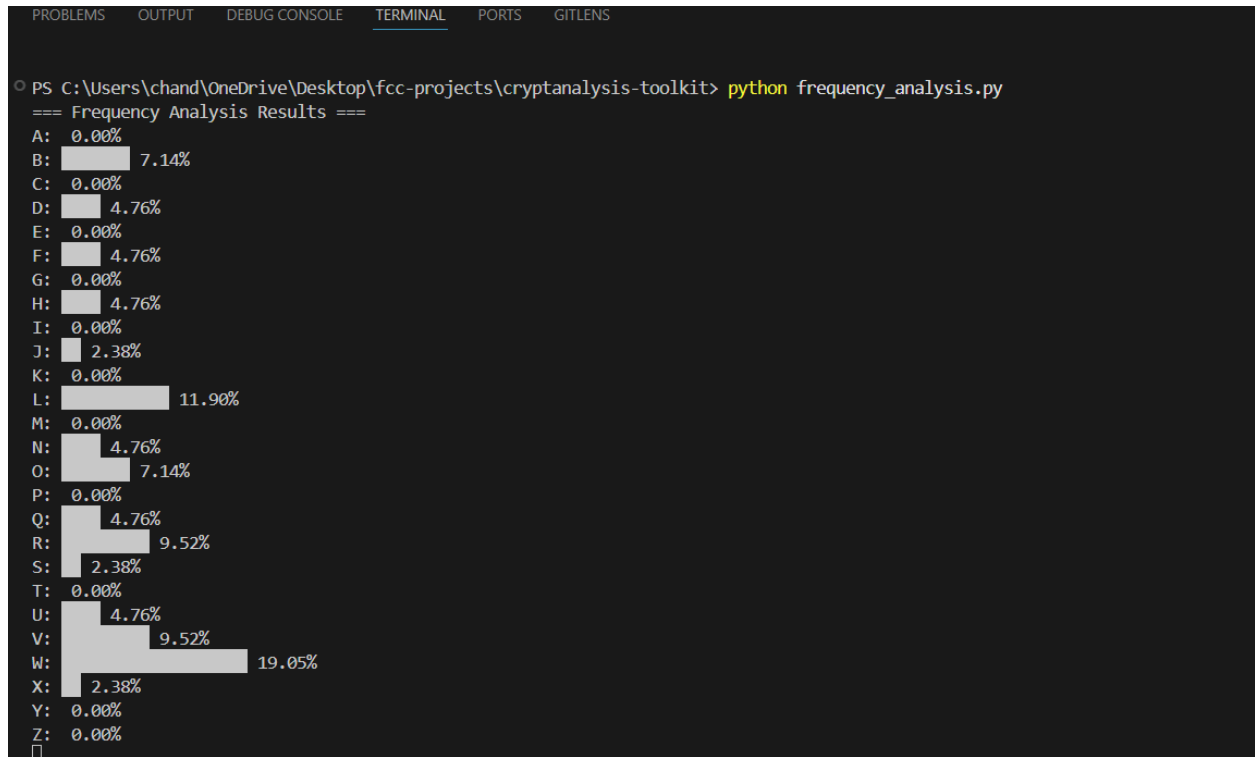
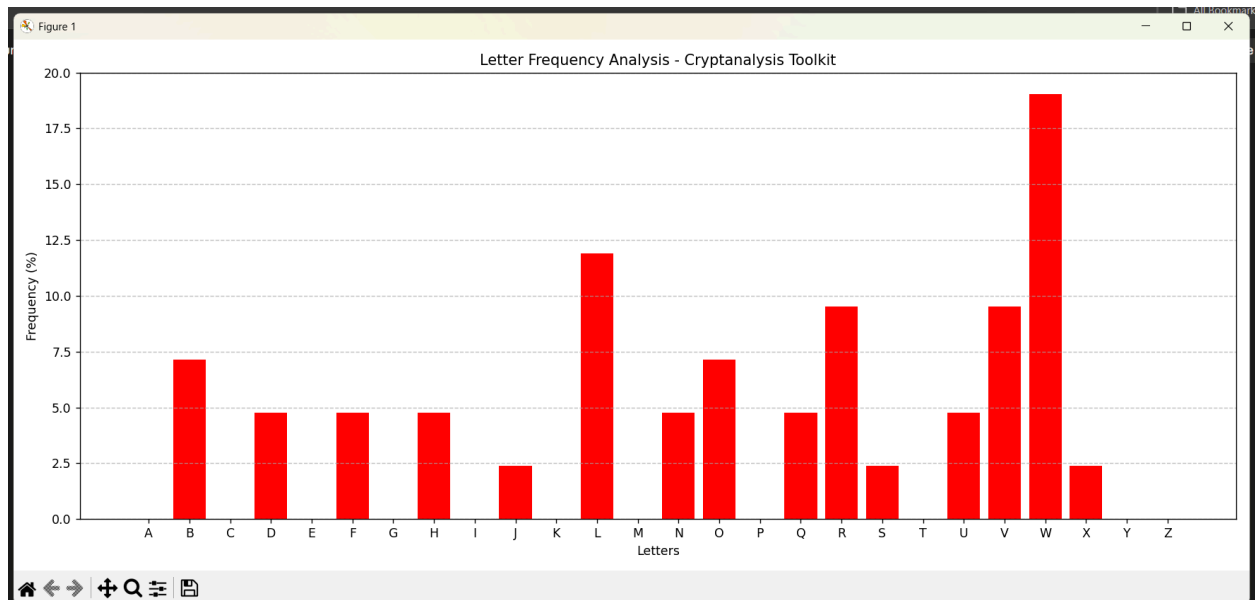


Fig 3: Matplotlib Bar Chart Visualization



Creativity Reflections

The frequency analysis visualizer extends the toolkit with a real cryptanalysis attack technique. By plotting letter distributions in ciphertext, patterns become visible that can be exploited to break classical ciphers. This demonstrates how attackers identify weaknesses in substitution ciphers, reinforcing the importance of stronger modern encryption algorithms.